

APAPO

Universal Hypergrid Platform

Software Design Document

Part II — Deep Implementation Reference

Attribute	Value
Document	Apapo SDD Part II — Deep Implementation Reference
Companion	Apapo SDD Part I (system overview, domain operating systems, deployment topology)
Version	1.0 — March 2026
Status	Authoritative Reference — Active Design
Audience	Platform engineers · Core contributors · SDK authors · System integrators · Domain system builders
Scope	Complete internal implementation details for: HyperCell model · Hypercube internals · Hypergraph engine · HyperQL parser and planner · HG-CRDT merge engine · HG-COMP formula and AI systems · HG-VIEW engine · HG-SPACE/HG-NS · HG-ID multi-identity · HG-PLUGIN interface · HG-EXPORT · HG-OPS · Full persistence schema · Security pipeline · Domain-layer bridge pattern · Kogi/Ume/Qala internal data schemas · WritebackService · kogi-engine sub-systems · Apapo runtime environment
Classification	Confidential — Internal Use Only

Table of Contents

Chapter 1 — Grid and Hypercube: Core Data Structures

1.1 The Grid — Root Runtime Container

A Grid is the root runtime container for one complete Hypergrid deployment. It owns all Hypercubes, the Hypergraph, the EventLog, the CrdtLog, the NamespaceRegistry, all Spaces, all federation peer relationships, and the plugin registry. Each domain operating system (Kogi, Ume, Qala) is a Grid deployment. Multiple Grids can federate via CrossGridLink edges. Within a single Grid, all data shares one Universal Cell Store, one EventLog, and one Hypergraph.

```
// hypergrid::core::Grid - root runtime container (Rust)
pub struct Grid {
    pub grid_id:      GridId,          // UUID - globally unique, immutable
    pub name:         String,
    pub version:      semver::Version, // Hypergrid substrate version
    pub namespace_path: NamespacePath, // hypergrid://{grid_name}/

    // — Registered Hypercubes —
    pub cubes:        HashMap<CubeId, Arc<RwLock<Hypercube>>>,
    pub cube_name_index: HashMap<String, CubeId>, //
    "kogi.portfolio.components" → CubeId
    pub cube_ns_index:  HashMap<NamespacePath, CubeId>,

    // — Core infrastructure —
    pub hypergraph:     Arc<RwLock<Hypergraph>>,
    pub event_log:      Arc<EventLog>,
    pub crdt_log:       Arc<CrdtLog>,
    pub vector_clock:   Arc<Mutex<VectorClock>>,

    // — Spatial systems —
    pub space_registry: Arc<SpaceRegistry>,
    pub workspace_store: Arc<WorkspaceStore>,
    pub namespace_reg:  Arc<NamespaceRegistry>,

    // — Identity & permissions —
    pub identity_store: Arc<IdentityStore>,
    pub policy_engine:  Arc<PolicyEngine>,

    // — Federation —
    pub federation_peers: Vec<FederationPeer>,
    pub node_id:         NodeId,          // this node's ID in VectorClock

    // — Plugin registry —
    pub plugin_registry: Arc<PluginRegistry>,

    // — Storage backend (pluggable) —
    pub cell_store:      Arc<dyn CellStoreBackend + Send + Sync>,
    pub snapshot_store:  Arc<dyn SnapshotBackend + Send + Sync>,
}

// CellStoreBackend - the pluggable physical storage interface
```

```

pub trait CellStoreBackend: Send + Sync {
    fn get_cell(&self, grid_id: GridId, coord: &DimCoordinate) ->
    Option<HyperCell>;
    fn put_cell(&self, cell: HyperCell) -> Result<(), StoreError>;
    fn put_cell_batch(&self, cells: Vec<HyperCell>) -> Result<(), StoreError>;
    fn delete_cell(&self, coord: &DimCoordinate) -> Result<(), StoreError>;
    fn scan_slice(&self, cube_id: CubeId, slice: &DimSlice, limit: u32, offset:
    u32)
        -> Vec<HyperCell>;
    fn scan_row(&self, cube_id: CubeId, dl_key: &DimKey)
        -> Vec<HyperCell>; // All cells for one HyperRow
    fn apply_crdt_op(&self, op: &CrdtOperation)
        -> Result<CrdtResult, CrdtError>;
    fn fold(&self, cube_id: CubeId, fold_spec: &DimFoldSpec)
        -> Vec<AggregateRow>;
    fn estimate_cardinality(&self, cube_id: CubeId, axis_id: AxisId)
        -> u64;
}

```

1.2 Hypercube — Complete Internal Structure

A Hypercube is the N-dimensional generalization of a spreadsheet sheet. It is a schema-plus-metadata container — the actual cell data lives in the Grid's CellStoreBackend. All domain systems register one Hypercube per major entity type using the naming convention {domain}.{entity_type}.

```

pub struct Hypercube {
    pub cube_id:      CubeId,
    pub grid_id:      GridId,
    pub name:         String, // e.g. "kogi.portfolio.components"
    pub namespace_path: NamespacePath,
    pub space_id:     Option<SpaceId>,

    // — Dimension axis definitions (ordered D1..Dn) —————
    pub dims:         Vec<DimensionAxis>,
    pub n:            u8, // 2 ≤ n ≤ 16

    // — Attribute schema —————
    pub attr_registry: AttributeKeyRegistry,
    // All attribute keys every HyperCell in this cube may carry

    // — Storage encoding (set at creation, immutable) —————
    pub encoding:      StorageEncoding,
    // Dense | Sparse | Hybrid { dense_dims: Vec<u8> }

    // — CRDT config per attribute key —————
    pub crdt_config:   HashMap<AttributeKey, CrdtSemantics>,

    // — Plugin bindings —————
    pub plugin_ids:    Vec<PluginId>,

    // — Governance —————
    pub governance:    CubeGovernance,
    pub visibility:     CubeVisibility, // Public|Tenant|Identity|Private
}

```

```

    pub policy_ids:          Vec<PolicyId>,

    // — Lifecycle —————
    pub status:              CubeStatus,    // Active|Archived|Migrating
    pub schema_version:      u32,
    pub created_at:          DateTime<Utc>,
    pub updated_at:          DateTime<Utc>,

    // — Computed attribute cache config —————
    pub computed_cache_ttl: Duration,
    pub computed_cache:      Arc<ComputedAttrCache>,
}

pub struct DimensionAxis {
    pub axis_id:             AxisId,
    pub dim_index:           u8,            // 1=D1, 2=D2, ... 16=D16
    pub name:                String,        // "rows", "columns", "time", "region",
    "tenant"...
    pub axis_type:           AxisType,      //
    EntityAxis|PropertyAxis|TimeAxis|GeoAxis|...
    pub key_type:            KeyType,       //
    Uuid|String|Integer|Timestamp|GeoHash|...
    pub key_cardinality:     KeyCardinality, // Dense(max_keys)|Sparse|Infinite
    pub ordering:            AxisOrdering,  //
    Unordered|Lexicographic|Chronological|Numeric|Custom
    pub nullable:           bool,
    pub visibility:          AxisVisibility,
    pub index_strategy:      IndexStrategy, //
    BTree|Hash|GIN|BRIN|TimeSeries|Spatial|ZOrder
    pub hierarchy:           Option<HierarchyDef>,
    pub crdt_semantics:      CrdtSemantics, // default CRDT for this whole axis
    pub plugin_id:          Option<PluginId>,
}

```

1.3 HyperCell — Complete Data Model

The HyperCell is the atomic data unit of the entire Hypergrid system. It lives at a specific N-dimensional coordinate and carries an N-attribute map. Every domain entity field, every AI-computed signal, every governance flag, every relationship reference is an attribute in a HyperCell. The HyperCell is the universal container for all structured information in Apapo.

```

pub struct HyperCell {
    // — Coordinate —————
    pub grid_id:             GridId,
    pub cube_id:             CubeId,
    pub coord:               DimCoordinate,
    // coord.keys: Vec<DimKey> — one key per dimension, ordered D1..DN
    // N=2 example: [entity_uuid, "health_score"]
    // N=4 example: [entity_uuid, "health_score", "2026-Q2", "EMEA"]

    // — Payload —————
    pub attributes:          AttributeMap,
    // HashMap<AttributeKey, TypedAttrValue>
    // Always includes built-ins: "value", "data_type", "format"
}

```

```

// Domain keys registered in AttributeKeyRegistry

// — Versioning & causality —
pub vector_clock: VectorClock, // HashMap<NodeId, u64>
pub version: u64, // monotonic per-cell counter
pub created_at: DateTime<Utc>,
pub updated_at: DateTime<Utc>,
pub last_actor: ActorId, // "{tenant_id}:{identity_tag}:{node_id}"

// — Governance —
pub visibility: CellVisibility, // Public|Tenant|Identity|Private
pub policy_ids: Vec<PolicyId>,
pub audit_trail: EventLogRef, // pointer to owning EventLog entry
sequence

// — Graph integration —
pub edge_refs: Vec<EdgeRef>, // edges originating from this cell

// — Computed attribute cache —
pub computed_cache: HashMap<AttributeKey, CachedComputedAttr>,
// CachedComputedAttr { value, computed_by, computed_at, ttl, confidence }
}

pub struct DimCoordinate {
    pub keys: Vec<DimKey>, // ordered [D1_key, D2_key, ..., Dn_key]
    pub n: u8, // must match parent Hypercube.n
}

pub enum DimKey {
    Uuid(Uuid),
    Text(String),
    Integer(i64),
    Timestamp(DateTime<Utc>),
    GeoHash(GeoHash),
    HierarchyPath(Vec<String>),
    SemVer(semver::Version),
    TenantId(TenantId),
    Null,
}

// TypedAttrValue — all possible values in an AttributeMap
pub enum TypedAttrValue {
    Text(String),
    Number(f64),
    Integer(i64),
    Currency { amount: Decimal, currency_code: String },
    Percent(f64),
    Bool(bool),
    Date(NaiveDate),
    DateTime(DateTime<Utc>),
    Duration(Duration),
    Relation(EntityRef),
    MultiRelation(Vec<EntityRef>),
    User(IdentityId),
    MultiUser(Vec<IdentityId>),
    NamespaceRef(NamespacePath),
    Enum { variant: String, enum_type: String },
    MultiEnum(Vec<String>),
    Tag(String),
    TagSet(HashSet<String>),
    Json(serde_json::Value),
}

```

```

    Formula { expression: String, result: Box<TypedAttrValue>, cached_at:
DateTime<Utc> },
    AISignal {
        value:          Box<TypedAttrValue>,
        model_id:       String,
        computed_at:    DateTime<Utc>,
        confidence:     f64,                // 0.0-1.0
        explanation:    Option<String>,
    },
    AnomalySignal {
        kind:           AnomalyKind,        //
SuddenChange|PatternBreak|OutlierValue|StateChange
        severity:      f64,
        baseline:      Box<TypedAttrValue>,
        deviation:     f64,
    },
    EdgeRef(EdgeId),
    ShadowRef { source_grid: GridId, source_coord: DimCoordinate },
    VectorClockSnapshot(VectorClock),
    EventLogRef(Vec<EventId>),
    RichText { markdown: String, plain: String },
    Attachment { url: String, mime_type: String, size_bytes: u64 },
    Custom { type_id: String, payload: serde_json::Value },
    Null,
}

```

1.4 AttributeKeyRegistry — Column Schema

The AttributeKeyRegistry is the schema definition layer for a Hypercube. Every attribute key any HyperCell may carry must be pre-registered here with its type, CRDT semantics, visibility rules, permission requirements, and optional computation. This is the strongly-typed column schema generalized to apply across all N dimensions of the cube.

```

pub struct AttributeKeyRegistry {
    pub entries: IndexMap<AttributeKey, AttributeKeyDef>,
    // IndexMap preserves insertion order for column ordering in views
}

pub struct AttributeKeyDef {
    pub key:          AttributeKey,        // unique string within this cube
    pub display_name: String,
    pub attr_type:     AttributeType,      //
Text|Number|Currency|Percent|Bool|Date|DateTime|
    //
Duration|Enum|MultiEnum|Relation|MultiRelation|
    //
User|Tag|Json|Computed|AI|Formula|Audit|Custom
    pub description:  String,
    pub dimension_scope: DimScope,        // AllDimensions |
SpecificAxes(Vec<AxisId>)
    pub default_value: Option<TypedAttrValue>,
    pub required:      bool,
    pub visibility:     AttrVisibility,    // Public|Tenant|Identity|Owner
    pub write_permission: PermissionTier, // min tier to write this attr
}

```

```
pub crdt_semantics: CrdtSemantics, //
LWW|OR-Set|Counter|Max|Min|Lattice|AppendLog|Custom
pub computation: Option<AttrComputation>,
pub aggregation: Vec<AggFn>, //
SUM|AVG|MIN|MAX|COUNT|MEDIAN|STDDEV|...
pub index_strategy: IndexStrategy, // None|BTree|Hash|GIN|BRIN
pub searchable: bool,
pub version_track: bool, // track full history in EventLog?
pub inherits_from: Option<AttributeKey>, // hierarchy inheritance
pub export_label: Option<String>, // override in CSV/XLSX headers
pub plugin_id: Option<PluginId>,
pub notes: String, // developer documentation
}

pub enum AttrComputation {
  None,
  Formula { expression: FormulaExpr },
  AiEngine {
    engine_id: AIEngineId,
    model_name: String,
    parameters: HashMap<String, serde_json::Value>,
    input_attrs: Vec<AttributeKey>,
    staleness_ttl: Duration,
  },
  Rollup { axis_id: AxisId, agg_fn: AggFn, source_attr: AttributeKey },
  DimFold { axis_id: AxisId, agg_fn: AggFn },
  GraphAggregate {
    edge_type: EdgeType,
    direction: EdgeDirection,
    agg_attr: AttributeKey,
    agg_fn: AggFn,
  },
}
```

1.5 Storage Encoding Strategies

Hypercubes choose their storage encoding at creation time based on expected data density. The encoding determines physical storage layout and index strategy in the CellStoreBackend. All encodings produce identical logical query results — the difference is performance and storage cost.

Encoding	When To Use	Physical Schema	Lookup Perf	Storage Cost
Dense	N=2 with small, fully-populated key sets. Every row×column pair has a value. All conventional 2D entity×field cubes.	PostgreSQL 2D row-per-entity layout. attr_map stored as JSONB column. $O(D_1 \cdot \text{card} \times D_2 \cdot \text{card})$ rows.	$O(1)$ primary key. Range scans contiguous. SIMD-friendly aggregations.	$O(D_1 \cdot \text{card} \times D_2 \cdot \text{card})$ — predictable fixed cost
Sparse	Any axis with cardinality > 100K. TimeAxis, GeoAxis, GraphAxis. Most	Key-value table: $(k_1, k_2, \dots, k_n) \rightarrow \text{attr_map}$ in a hash-indexed table. Only non-null cells stored.	$O(\log n)$ via composite BTree/BRIN	$O(\text{non-null cells})$ — scales with data, not with key space

Encoding	When To Use	Physical Schema	Lookup Perf	Storage Cost
	coordinate combos have no value.		index on dim key columns.	
Hybrid	$D_1 \times D_2$ dense (entity $\times$ property field); $D_3 - D_n$ sparse (time, region, tenant). Most common production pattern.	Dense 2D base table for $D_1 \times D_2$. Separate sparse key-value extension table for $D_3 - D_n$ cells. Two physical tables joined at query time.	$O(1)$ for base 2D queries. $O(\log n)$ for higher-dim extension queries.	Dense base cost + sparse extension. Best of both worlds.

Implementation Note: Use Hybrid for ALL domain entity cubes (kogi.portfolio.components, ume.kernel.modules, qala.solutions). The entity $\times$ field $D_1 \times D_2$ slice is dense and hot-path. Time, category, and tenant dimensions are sparse. Using Dense encoding for a TimeAxis would pre-allocate billions of empty cells.

1.6 Schema Evolution — Live Zero-Downtime Modifications

Hypercubes evolve without downtime. Additive schema changes are CRDT-commutative — all federation peers safely apply them independently and converge. Destructive changes require a governance gate.

Operation	Safety Class	Protocol	Federation-Safe?	Downtime?
Add new DimensionAxis (D_{n+1})	Safe — additive	Register axis in cube schema. Existing cells gain new dim with null/default key. New cells use new dim immediately.	Yes — additive SchemaEvent commutes	None
Add new AttributeKeyDef	Safe — additive	Register new key in AttributeKeyRegistry. Existing cells return default_value for new key. No migration.	Yes — commutes	None
Expand enum variant set	Safe — additive	Add variant to EnumDef. Existing values remain valid. New variant available immediately.	Yes — commutes	None
Rename attribute key	Safe with alias	Add alias from old key to new key. Old key remains readable; writes go to new key. Remove alias post-migration.	Yes — alias commutes	None
Change CRDT semantics	Risky — careful	Schema version bump. Existing values unchanged. New semantics apply only to mutations after schema version change. No retroactive merge.	Caution: inflight concurrent ops may misbehave during	None

Operation	Safety Class	Protocol	Federation-Safe?	Downtime?
			version transition	
Remove attribute key	Destructive — governance gate	Requires GovernanceProposal vote. Key tombstoned (not deleted from EventLog). Historical values preserved for AS_OF queries.	Yes — after vote applied	None (post-approval)
Remove DimensionAxis	Destructive — governance gate	GovernanceProposal vote required. All cells on removed axis tombstoned. EventLog preserves full history.	Yes — after vote applied	Brief recommended
Change axis KeyType	Breaking — migration required	Add new axis; old axis migrated via background job. Dual-write window. Requires FeatureFlagController.	No — requires coordinated migration	Migration window

Chapter 2 — HG-CRDT: Distributed Consistency Engine

2.1 Three-Level CRDT Architecture

HG-CRDT operates at three distinct granularity levels simultaneously. Every implementor must understand all three levels to build correct federation, multi-user editing, and schema evolution.

Level	Scope	Operation Class	Merge Strategy	Storage Location
Grid-level	Full Grid sync across federation peers	Gossip: each peer maintains VectorClock per grid; delta sync on 30s heartbeat	VectorClock element-wise max; delta replay in causal order	CrdtLog table (PostgreSQL) + Redis hot buffer
Hypercube-level	Schema: dimension additions, attribute registrations	SchemaEvent log: ordered, idempotent. Non-conflicting if additive.	All additive changes commute. Destructive changes blocked by governance gate.	SchemaEventLog table (PostgreSQL append-only)
Cell-attribute-level	Individual TypedAttrValue mutations	Per-attribute CRDT semantics. Each AttributeKeyDef has independent CrdtSemantics.	LWW OR-Set Counter Lattice AppendLog JSON-deep Custom	CellStore (PostgreSQL) + CrdtLog delta

2.2 CrdtOperation — The Fundamental Mutation Unit

Every state change in Hypergrid is expressed as a CrdtOperation. This is the unit of federation delta sync, the unit of the CrdtLog, and the unit of conflict detection. All writes — users, AI engines, federation sync, schema migrations — produce CrdtOperations.

```
pub enum CrdtOperation {
  // — Cell attribute mutations
  SetAttr {
    coord:      DimCoordinate,
    attr_key:    AttributeKey,
    value:       TypedAttrValue,
    timestamp:   VectorClock,      // LWW tiebreak basis
    actor:       ActorId,
  },
  AddToSet {
    coord:      DimCoordinate,
    attr_key:    AttributeKey,
```

```

        element: TypedAttrValue,
        unique_tag: Uuid,           // OR-Set element tag
        actor: ActorId,
    },
    RemoveFromSet {
        coord: DimCoordinate,
        attr_key: AttributeKey,
        unique_tag: Uuid,           // must match a previous AddToSet tag
        actor: ActorId,
    },
    IncrCounter {
        coord: DimCoordinate,
        attr_key: AttributeKey,
        delta: i64,                 // GrowOnly: must be > 0; PN: any sign
        node_id: NodeId,
    },
    TransitionState {
        coord: DimCoordinate,
        attr_key: AttributeKey,
        from_state: String,
        to_state: String,
        actor: ActorId,
        timestamp: VectorClock,     // Lattice merge basis
    },
    AppendLog {
        coord: DimCoordinate,
        attr_key: AttributeKey,
        entry: TypedAttrValue,
        causal_ts: VectorClock,     // ordering for Append-only attr log
    },
    DeepMergeJson {
        coord: DimCoordinate,
        attr_key: AttributeKey,
        delta: serde_json::Value,   // only changed leaf keys
        timestamp: VectorClock,
        actor: ActorId,
    },

    // — Schema mutations (Hypercube-level) —————
    AddDimension { cube_id: CubeId, axis: DimensionAxis, schema_ts: u64 },
    RegisterAttrKey { cube_id: CubeId, attr_def: AttributeKeyDef, schema_ts: u64 },
},
    TombstoneAttrKey { cube_id: CubeId, attr_key: AttributeKey, schema_ts: u64 },

    // — Graph mutations —————
    AddEdge { edge: HypergraphEdge },
    RemoveEdge { edge_id: EdgeId },
    UpdateEdgeAttr { edge_id: EdgeId, attr_key: AttributeKey, value:
TypedAttrValue, ts: VectorClock },

    // — Identity / namespace —————
    RegisterNamespace { path: NamespacePath, entity_ref: EntityRef },
    AddFederationPeer { peer: FederationPeer },
}

```

2.3 Merge Engine — apply_crdt_op()

The merge engine is the heart of HG-CRDT. It applies incoming `CrdtOperations` from any source (local user, federation peer, AI writeback, migration) in a way that preserves all CRDT invariants. The function is deterministic: the same set of operations applied in any order always produces the same result.

```
impl CrdtMergeEngine {
  pub fn apply_crdt_op(
    &self,
    op: &CrdtOperation,
    store: &mut dyn CellStoreBackend,
    event_log: &mut EventLog,
  ) -> Result<MergeResult, CrdtError> {
    match op {
      CrdtOperation::SetAttr { coord, attr_key, value, timestamp, actor } =>
      {
        let crdt = self.registry.crdt_semantics_for(coord.cube_id,
attr_key)?;
        match crdt {
          // — LastWriteWins —
          CrdtSemantics::LastWriteWins => {
            let current_ts = store.get_cell(coord.grid_id, coord)
              .and_then(|c| c.attributes.get(attr_key))
              .and_then(|a| a.vector_clock());
            if timestamp.dominates(&current_ts.unwrap_or_default()) {
              store.put_attr(coord, attr_key, value.clone())?;
              event_log.append(EventEntry::set_attr(op))?;
              Ok(MergeResult::Applied)
            } else {
              // Losing write: record for conflict surfacing
              event_log.append(EventEntry::conflict(op,
ConflictKind::LwwLosing))?;
              Ok(MergeResult::ConflictRecorded { kept:
"current".into() })
            }
          }
          // — Status Lattice —
          CrdtSemantics::Lattice(lattice_def) => {
            let current_state = store.get_attr_string(coord, attr_key)
              .unwrap_or_default();
            let new_state = value.as_str().unwrap_or_default();
            let merged = lattice_def.join(&current_state, new_state);
            // Lattice join = least upper bound: always advances
            state, never regresses
            if merged != current_state {
              store.put_attr(coord, attr_key,
TypedAttrValue::Text(merged))?;
              event_log.append(EventEntry::set_attr(op))?;
            }
            Ok(MergeResult::Applied)
          }
        }
        _ => self.apply_generic(crdt, op, store, event_log),
      }
    }

    CrdtOperation::AddToSet { coord, attr_key, element, unique_tag, .. }
    => {
      // OR-Set: always add the element with its unique_tag.
      // Concurrent adds to same logical value with different tags: BOTH
      survive.
    }
  }
}
```

```

        store.or_set_add(coord, attr_key, element.clone(), *unique_tag)?;
        event_log.append(EventEntry::or_set_add(op))?;
        Ok(MergeResult::Applied)
    }

    CrdtOperation::RemoveFromSet { coord, attr_key, unique_tag, .. } => {
        // OR-Set: remove ONLY the entry with this exact unique_tag.
        // Concurrent adds of the same logical value with different tags
survive.
        store.or_set_remove(coord, attr_key, *unique_tag)?;
        event_log.append(EventEntry::or_set_remove(op))?;
        Ok(MergeResult::Applied)
    }

    CrdtOperation::IncrCounter { coord, attr_key, delta, node_id } => {
        // GrowOnly counter: delta must be > 0. PN-Counter: any sign.
        let crdt = self.registry.crdt_semantics_for(coord.cube_id,
attr_key)?;
        match crdt {
            CrdtSemantics::GrowOnlyCounter => {
                if *delta <= 0 { return Err(CrdtError::GrowOnlyViolation); }

                store.counter_incr(coord, attr_key, *delta, *node_id)?;
            }
            CrdtSemantics::PNCounter => {
                store.counter_incr(coord, attr_key, *delta, *node_id)?;
            }
            _ => return Err(CrdtError::WrongCrdtType),
        }
        event_log.append(EventEntry::counter_incr(op))?;
        Ok(MergeResult::Applied)
    }

    CrdtOperation::TransitionState { coord, attr_key, from_state,
to_state, .. } => {
        // Lattice: verify transition is valid in partial order.
        let crdt = self.registry.crdt_semantics_for(coord.cube_id,
attr_key)?;
        if let CrdtSemantics::Lattice(lattice) = crdt {
            if !lattice.is_valid_transition(from_state, to_state) {
                return Err(CrdtError::InvalidStateTransition {
                    from: from_state.clone(), to: to_state.clone()
                });
            }
            let current = store.get_attr_string(coord,
attr_key).unwrap_or_default();
            let merged = lattice.join(&current, to_state);
            if merged != current {
                store.put_attr(coord, attr_key,
TypedAttrValue::Text(merged))?;
                event_log.append(EventEntry::state_transition(op))?;
            }
            Ok(MergeResult::Applied)
        } else {
            Err(CrdtError::WrongCrdtType)
        }
    }

    _ => self.dispatch_specialized(op, store, event_log),
}
}

```

}

2.4 VectorClock — Causal Ordering

The VectorClock is a `HashMap<NodeId, u64>`. Every `CrdtOperation` carries the issuing node's VectorClock. Receivers use it to determine whether operations are causally ordered, concurrent, or duplicated. NodeIds are formatted as `"{federation_node_id}:{identity_tag}"` to carry identity context alongside causal context.

```
pub struct VectorClock(pub HashMap<NodeId, u64>);

impl VectorClock {
    // Advance this node's logical clock before any local mutation
    pub fn tick(&mut self, node_id: NodeId) -> u64 {
        let counter = self.0.entry(node_id).or_insert(0);
        *counter += 1;
        *counter
    }

    // Merge two clocks: element-wise maximum (call on receiving remote ops)
    pub fn merge(&mut self, other: &VectorClock) {
        for (node, &ts) in &other.0 {
            self.0.entry(*node)
                .and_modify(|v| *v = (*v).max(ts))
                .or_insert(ts);
        }
    }

    // Does self causally dominate other? (self happened-after or equal)
    pub fn dominates(&self, other: &VectorClock) -> bool {
        other.0.iter().all(|(node, &ts)| self.0.get(node).copied().unwrap_or(0) >=
ts)
    }

    // Are these clocks concurrent? (neither happened-before the other)
    pub fn concurrent_with(&self, other: &VectorClock) -> bool {
        !self.dominates(other) && !other.dominates(self)
    }

    // happened_before: self causally precedes other
    pub fn happened_before(&self, other: &VectorClock) -> bool {
        self.0.iter().all(|(n, &ts)| other.0.get(n).copied().unwrap_or(0) >= ts)
        && self.0 != other.0
    }

    // Delta: which (node, ts) pairs in self are newer than other?
    // Used to compute what to send in federation delta sync
    pub fn delta_since(&self, other: &VectorClock) -> HashMap<NodeId, u64> {
        self.0.iter()
            .filter(|(node, &ts)| ts > other.0.get(node).copied().unwrap_or(0))
            .map(|(&node, &ts)| (node, ts))
            .collect()
    }
}
```

```
// NodeId format for identity-aware CRDT:
// "{federation_node_id}:{identity_tag}"
// Example: "node-us-east-1:@jordan-dev"
// This ensures identity-tagged ops carry the correct causal context
// and enables per-identity conflict tracking
```

2.5 Per-Attribute CRDT Reference

CRDT Type	Merge Rule	Conflict Behavior	Typical Attributes
LastWriteWins (LWW)	Highest VectorClock timestamp wins. Tiebreak: ActorId lexicographic sort.	Losing write stored in ConflictRecord; surfaced to actor via notification. Oba logs conflict.	name, description, title, config, JSON blobs, enum fields, reference IDs, all human-authored fields
OR-Set	All concurrent Adds survive. Removes are tagged: only the exact tagged entry is removed.	No conflict for concurrent Adds. Remove vs concurrent Add: Add wins (OR-Set semantics).	tags, owners, policy_ids, toolbox_ids, dependency_ids, member_lists, identity_tags, child_ids
GrowOnlyCounter	Sum all increment ops from all nodes. Never decrement.	Always commutes. No conflict possible. Any decrement op is rejected as CrdtError.	restart_count, view_count, like_count, follower_count, contribution_count, link_count
PN-Counter	Sum of (positive_ops – negative_ops) per node across all nodes.	Commutes. No conflict. Allows net decrease via tracked negative deltas.	budget_spent (tracked transactions), hours_logged, allocated_units
MaxRegister	Highest numeric value wins, regardless of causal ordering.	Always resolves to maximum. No human review needed.	version, sequence_number, sprint_number, generation, schema_version
MinRegister	Lowest value wins. Useful for earliest-deadline semantics.	Always resolves to minimum.	earliest_deadline, original_creation_timestamp, min_threshold
Status Lattice	Merge to least upper bound in the configured partial order. No backward transitions.	Governance gate for transitions requiring approval. Concurrent contradictory transitions resolve to the higher lattice state.	status, lifecycle_state, state, phase — all governed lifecycle fields
Append-only Log	All appends commute. Causally ordered by VectorClock.	No conflict: all appends survive. Causal order guaranteed.	EventLog entries, comment threads, change_history, audit notes

CRDT Type	Merge Rule	Conflict Behavior	Typical Attributes
JSON CRDT	Deep-merge: recursive LWW per leaf JSON key.	Key-level conflicts use LWW. Structural conflicts (key type change) surface as ConflictRecord.	JSON configuration blobs, settings objects, structured payloads
Custom (Plugin)	Plugin provides merge(a, b, ctx) → TypedAttrValue.	Plugin-defined: may surface ConflictRecord for human review, auto-resolve, or reject.	Domain-pack fields, third-party attribute types registered via HG-PLUGIN AttributeTypePlugin

2.6 Federation Sync Protocol — Complete 8-Phase Sequence

The federation protocol is the complete mechanism for bringing two Grid nodes to mutual consistency. Implementors must handle all 8 phases correctly. Partial implementations will produce data loss or divergence under network partitions.

Phase	Direction	Exchange	Success Condition	Failure Mode
1. Discover y	Either peer initiates	POST /federation/peers { peer_url, grid_id, trust_level }. Mutual HMAC-SHA256 or mTLS challenge-response. Grid IDs exchanged.	Both peers have each other in federation_peers. VectorClock slot allocated.	Auth failure → reject. Retry with exponential backoff (1s, 2s, 4s...). Alert admin after 5 failures.
2. Handshake	Both peers	Exchange VectorClock state + schema_hash_by_cube + last_event_sequence. Compute what needs to sync.	Both peers know current VectorClock and schema versions for all shared cubes.	Exchange failure → retry after 5s. Log and alert after 3 retries.
3. Schema Sync	Both (if schema_hash differs)	GET /federation/schema/delta?since=schema_ts. Receive SchemaEvents missing from this peer. Apply AddDimension, RegisterAttrKey ops idempotently.	All schema versions aligned. Same AttributeKeyRegistry contents at both peers.	Schema conflict: additive always accepted. Breaking changes blocked; admin alert required.
4. Cell Delta Sync	Both peers	delta_since(peer_vc) computes op set. Batch in 10K-op chunks. Compress (Zstd level 3). POST /federation/crdt/sync. Receive per-batch ack.	Peer VectorClock advances to cover all local operations. apply_crdt_op() applied for all received ops.	Batch failure → resend. Maintain causal in-order delivery per peer chain. Max 3 retries per batch before alert.
5. Conflict Surface	Receiver → origin	ConflictRecords generated by apply_crdt_op() sent back to origin: POST	Origin has all ConflictRecords. Routes to affected	Best-effort delivery. Conflicts also surfaced locally on next read of affected cell.

Phase	Direction	Exchange	Success Condition	Failure Mode
		/federation/conflicts { records }	actors via notification service.	
6. Namespace Sync	Initiating peer	Exchange NamespaceRegistry entries for entities reachable via CrossGridLink (only entries with federate=true).	All cross-linked entities resolvable at both peers.	At-least-once delivery. Registry ops idempotent; duplicate entries safe to ignore.
7. Heartbeat	Both peers	Every 30s: GET /federation/heartbeat → { node_id, vector_clock, health_status }. Trigger full delta sync if VectorClock diverged since last heartbeat.	VectorClock stays ≤ 30s behind at steady state.	3 missed heartbeats → peer marked Degraded. Shadow cell sync suspended. Alert dispatched.
8. Shadow Sync	Event-driven (ongoing)	Kafka topic: shadow_sync.{source_grid_id}. Producer: source grid on every MirrorAttribute change. Consumer: all subscribing host grids.	Shadow cells updated < 1s after source mutation at steady state.	Consumer lag > 5s → alert. Lag > 60s → shadow cells marked shadow_status=Degraded. Backpressure applied.

Chapter 3 — HG-GRAPH: Hypergraph Implementation

3.1 Hypergraph Data Structures

The Hypergraph is a first-class component of every Grid deployment. Every entity in every Hypercube is simultaneously a node in the Hypergraph. Edges are typed, directed, weighted, and carry their own AttributeMaps. The Hypergraph uses the same CrdtLog for consistency and the same EventLog for audit as the cell store.

```
pub struct Hypergraph {
  pub graph_id:      GraphId,
  pub grid_id:       GridId,
  pub nodes:         NodeStore,           // NodeId → HypergraphNode
  pub edges:         EdgeStore,           // EdgeId → HypergraphEdge
  pub adj_index:     AdjacencyIndex,      // NodeId → (outbound_edges,
inbound_edges)
  pub subgraphs:     HashMap<SpaceId, Subgraph>, // Space-scoped, cached hourly
  pub link_forests:  HashMap<IdentityId, LinkForestCache>,
}

pub struct HypergraphNode {
  pub node_id:       NodeId,              // UUID
  pub node_type:     NodeType,            //
Cell|Row|Cube|Space|Grid|External|Custom
  pub entity_ref:    EntityRef,           //
CellRef|RowRef|CubeRef|SpaceRef|GridRef
  pub attributes:    AttributeMap,        // node-level attrs (reuses HG-CELL
model)
  pub namespace_path: Option<NamespacePath>,
  pub visibility:    NodeVisibility,
  pub identity_tag:  Option<IdentityTag>,
}

pub struct HypergraphEdge {
  pub edge_id:       EdgeId,
  pub from_node:     NodeId,
  pub to_node:       NodeId,
  pub edge_type:     EdgeType,
  pub direction:     EdgeDirection,       // Directed|Bidirectional
  pub weight:        f64,                 // 0.0-1.0 (used by Dijkstra path
queries)
  pub attributes:    AttributeMap,        // edge-level attributes

  // Consent & shadow cell config
  pub visibility:    EdgeVisibility,
  pub consent_status: ConsentStatus,      //
None|Pending|Accepted|Declined|Revoked
  pub consent_config: Option<ConsentConfig>, // mirrored_attrs, write_back_attrs,
dim_scope

  // Scope & lifecycle
```

```

pub space_scope: Option<SpaceId>,
pub shadow_cell_id: Option<ShadowCellId>,
pub expires_at: Option<DateTime<Utc>>,
pub created_at: DateTime<Utc>,
pub created_by: ActorId,
}

pub struct ConsentConfig {
pub mirrored_attrs: Vec<AttributeKey>, // attrs reflected in shadow
cell
pub write_back_attrs: Vec<AttributeKey>, // attrs host can write back
pub mirrored_dim_scope: Option<Vec<DimSlicePredicate>>, // which dim keys to
mirror
pub update_policy: UpdatePolicy, //
RealTime|Batched(n_hours)|Manual
pub consented_at: DateTime<Utc>,
pub consented_by: ActorId,
pub expiry: Option<DateTime<Utc>>,
}

```

3.2 Complete Edge Type Registry

EdgeType	Source	Target	Consent Required	Shadow Cell	Cycle Detection	Notes
Hierarchy	Any	Any	No	No	Yes — enforced	Parent-child containment. Enables ROLLUP aggregation and DRILLDOWN HierarchyAxis queries. Weight = 1.0.
Dependency	Row/Cell	Row/Cell	No	Optional	Yes — enforced	Blocking dependency: from_node cannot complete before to_node. Cross-grid dependencies supported.
Association	Any	Any	No	No	No	Soft lateral link. Informational reference. No blocking semantics. Use for OKR cascades, module relationships.
Contains	Space/Cube	Cube/Row	No	No	No	Ownership containment. Space or Cube owns/contains this entity.
Derives	Row	Row	No	Optional	No	Derivation/fork. Row A was derived from Row B. Used for solution versioning, document forking.
CrossGridLink	Row/Cell	Row/Cell	Yes — from target	Yes	No	Inter-Grid connection. Core link network atom. Consent gates attribute mirroring.

EdgeType	Source	Target	Consent Required	Shadow Cell	Cycle Detection	Notes
ShadowOf	ShadowRow	Source Row	System-managed	N/A	No	System edge marking a row as a shadow. Never user-created directly. Inverse of CrossGridLink.
SpaceMembership	Identity	Space	Depends on Space type	No	No	Identity is a member of Space. Role, contribution_weight, stake_pct stored on edge attributes.
Collaborates	Identity	Row/Cube	Yes — mutual	Yes	No	Bidirectional collaboration. Both parties must consent. MirrorColumns created for agreed attributes.
References	Cell/Row	Cell/Row	No	No	No	Citation or cross-reference. Lighter than Dependency. No blocking semantics.
FederationPeer	Grid	Grid	Grid-level trust	System	No	CRDT federation link between two Grid deployments. Managed by federation coordinator service.
Namespace Alias	Namespace	Namespace	No	No	No	One namespace path aliases another. Used for renames and redirects.
ComputedFrom	Cell	Cell(s)	No	No	No	Provenance: this cell was computed from these source cells. Lineage queries use this edge type.
SubscribesTo	Identity	Row/Cube/Space	No	Optional	No	Identity receives change notifications. Platform connects to notification service.
InvestedIn	Identity	Row/Space	Yes — from target	Yes	No	Economic investment. Platform-specific financial semantics.
Employs	Space/Identity	Identity	Yes — from worker	Yes	No	Employment/contracting. Worker consents with configured visible attrs.
Custom(String)	Any	Any	Plugin-defined	Plugin-defined	Plugin-defined	Platform-registered via HG-PLUGIN EdgeTypePlugin trait.

3.3 Graph Traversal Algorithms

HG-GRAPH traversal is a first-class HyperQL operation (TRAVERSE GRAPH clause). All traversal algorithms respect the caller's VisibilityMask — nodes and edges not visible to the

caller are silently excluded. The following shows complete implementations of the primary traversal modes:

```
impl HypergraphTraverser {
  // BFS — used for LinkForest, ego graph, Space subgraph
  pub fn bfs(
    &self,
    start: NodeId,
    config: BfsConfig, // max_depth, edge_type_filter, direction, visibility
  ) -> TraversalResult {
    let mut visited = HashSet::new();
    let mut queue = VecDeque::new();
    let mut result = TraversalResult::new();
    queue.push_back((start, 0u8));

    while let Some((node_id, depth)) = queue.pop_front() {
      if depth > config.max_depth || visited.contains(&node_id) { continue; }

      let node = match self.graph.nodes.get(&node_id) { Some(n) => n, None
=> continue };
      if !config.visibility.allows_node(node) { continue; }
      visited.insert(node_id);
      result.add_node(node.clone(), depth);

      let edges = self.graph.adj_index
        .edges_from(node_id, config.direction)
        .into_iter()
        .filter(|e| config.edge_filter.as_ref().map_or(true, |f|
f.matches(e)))
        .filter(|e| config.visibility.allows_edge(e));

      for edge in edges {
        result.add_edge(edge.clone());
        let next = match config.direction {
          TraversalDirection::Outbound => edge.to_node,
          TraversalDirection::Inbound  => edge.from_node,
          TraversalDirection::Both     => {
            if edge.from_node == node_id { edge.to_node } else {
edge.from_node }
          }
        };
        queue.push_back((next, depth + 1));
      }
    }
    result
  }

  // Dijkstra shortest path — for "degrees of separation" queries
  pub fn shortest_path(
    &self,
    from: NodeId, to: NodeId,
    edge_weight_fn: &dyn Fn(&HypergraphEdge) -> f64,
    visibility: &VisibilityMask,
  ) -> Option<Path> {
    let mut dist: HashMap<NodeId, f64> = HashMap::new();
    let mut prev: HashMap<NodeId, (NodeId, EdgeId)> = HashMap::new();
    let mut heap = BinaryHeap::new();

    dist.insert(from, 0.0);
  }
}
```

```

    heap.push((OrderedFloat(0.0f64), from));

    while let Some((cost, node)) = heap.pop() {
        if node == to { return Some(self.reconstruct_path(&prev, from, to)); }
        if cost.0 > dist.get(&node).copied().unwrap_or(f64::INFINITY) {
            continue; }

        for edge in self.graph.adj_index.edges_from(node,
            TraversalDirection::Outbound) {
            if !visibility.allows_edge(edge) { continue; }
            let new_cost = cost.0 + edge_weight_fn(edge);
            if new_cost <
                dist.get(&edge.to_node).copied().unwrap_or(f64::INFINITY) {
                dist.insert(edge.to_node, new_cost);
                prev.insert(edge.to_node, (node, edge.edge_id));
                heap.push((OrderedFloat(new_cost), edge.to_node));
            }
        }
    }
    None
}

// Cycle detection for Hierarchy and Dependency edges — enforced at write time
pub fn would_create_cycle(
    &self,
    from: NodeId, to: NodeId,
    edge_type: &EdgeType,
) -> bool {
    // A cycle exists if there is already a path from `to` back to `from`
    self.bfs(to, BfsConfig {
        max_depth: 64,
        edge_filter: Some(EdgeTypeFilter::exact(edge_type.clone())),
        direction: TraversalDirection::Outbound,
        visibility: VisibilityMask::system_wide(),
    }).contains_node(from)
}

// Louvain community detection (simplified for implementation reference)
pub fn detect_communities(&self, space_id: SpaceId) -> HashMap<NodeId,
CommunityId> {
    let subgraph = self.graph.subgraphs.get(&space_id)
        .expect("Subgraph must be materialized before community detection");
    // Louvain Phase 1: greedy local modularity maximization
    // Louvain Phase 2: aggregate communities into super-nodes
    // Result stored as computed attribute on each HyperRow node
    louvain::detect(subgraph)
}
}

```

3.4 Shadow Cell Provisioning Pipeline

When a CrossGridLink is created and consent obtained, the shadow cell pipeline creates a read-only reflection of the source entity in the host Grid. This multi-step async process must handle partial failures and consent revocations gracefully.

```

async fn provision_shadow_cell(
    edge: &HypergraphEdge,
    consent: &ConsentConfig,
    host_grid: &Grid,
    source_client: &dyn RemoteGridClient,
) -> Result<ShadowCellId, ShadowError> {
    // Step 1: Fetch current source entity state (consented attrs only)
    let source_row = source_client
        .get_hyper_row(edge.to_node.entity_ref(), &consent.mirrored_attrs)
        .await
        .map_err(|e| ShadowError::SourceFetchFailed(e))?;

    // Step 2: Apply mirrored_dim_scope filter if configured
    let scoped_cells = consent.mirrored_dim_scope.as_ref()
        .map(|scope| source_row.cells_matching_dim_scope(scope))
        .unwrap_or_else(|| source_row.all_cells());

    // Step 3: Create ShadowRow in host grid
    let shadow_id = host_grid.cell_store.create_shadow_row(ShadowRowSpec {
        shadow_id:      Uuid::new_v4(),
        source_grid_id: source_client.grid_id(),
        source_dl_key:  source_row.dl_key,
        source_edge_id: edge.edge_id,
        visible_attrs:  consent.mirrored_attrs.clone(),
        write_back_attrs: consent.write_back_attrs.clone(),
        update_policy:  consent.update_policy.clone(),
        initial_cells:  scoped_cells,
    }).await?;

    // Step 4: Register ShadowOf edge in Hypergraph
    host_grid.hypergraph.write().add_edge(HypergraphEdge {
        edge_id:      Uuid::new_v4(),
        edge_type:    EdgeType::ShadowOf,
        from_node:    shadow_id.into(),
        to_node:      edge.to_node,
        direction:    EdgeDirection::Directed,
        weight:       1.0,
        attributes:   AttributeMap::empty(),
        consent_status: ConsentStatus::None, // system edge, no consent needed
        ..Default::default()
    }).await?;

    // Step 5: Subscribe Kafka shadow_sync topic for ongoing delta sync
    host_grid.shadow_sync_consumer
        .subscribe(source_client.grid_id(), shadow_id, consent)
        .await?;

    Ok(shadow_id)
}

// Shadow sync consumer — processes incremental MirrorAttribute deltas from Kafka
async fn process_shadow_delta(
    delta: ShadowDelta,
    host_grid: &Grid,
) -> Result<(), ShadowError> {
    // Guard: CrossGridLink must still be active
    let edge = host_grid.hypergraph.read().get_edge(delta.source_edge_id)?;
    if edge.consent_status != ConsentStatus::Accepted {
        return Err(ShadowError::LinkRevoked);
    }
}

```



```
// Guard: changed attr must be in consented mirrored_attrs
let consent = edge.consent_config.as_ref().ok_or(ShadowError::NoConsent)?;
if !consent.mirrored_attrs.contains(&delta.attr_key) {
    return Err(ShadowError::AttrNotConsented);
}

// Resolve host Grid shadow coord from source edge + source coord
let shadow_coord = host_grid.cell_store
    .shadow_row_coord_for(delta.source_edge_id, &delta.source_coord)?;

// Apply as system-tier SetAttr on the shadow cell
host_grid.crdt_merge_engine.apply_crdt_op(
    &CrdtOperation::SetAttr {
        coord: shadow_coord,
        attr_key: delta.attr_key,
        value: delta.new_value,
        timestamp: delta.source_timestamp,
        actor: ActorId::shadow_sync(delta.source_grid_id),
    },
    &mut *host_grid.cell_store.write(),
    &mut *host_grid.event_log.write(),
)?;
Ok(())
}
```

Chapter 4 — HG-QL: HyperQL Query Engine

4.1 Complete EBNF Grammar

```
-- HyperQL Full Grammar (EBNF) v1.0

query      ::= SELECT sel FROM source where? fold? expand? group? order? limit?
options?

-- SELECT clause
sel        ::= "*" | sel_item ("," sel_item)*
sel_item   ::= attr_expr | agg_expr | formula_expr
              | "D" uint "." identifier          -- dimension key reference
              | "EXPAND" axis_id "AS" COLUMNS(" sel_item ")
              | sel_item "AS" alias

attr_expr  ::= attr_key | "cell[" coord_expr "]" attr_key
agg_expr   ::= agg_fn "(" (attr_key | sel_item) ")"
agg_fn     ::= "SUM" | "AVG" | "COUNT" | "COUNT_UNIQUE" | "MIN" | "MAX"
              | "MEDIAN" | "PERCENTILE" "(" float_lit ")"
              | "STDDEV" | "VAR" | "DISTRIBUTION"
              | "FOLD_SUM" | "FOLD_AVG" | "ROLLUP" | "DRILLDOWN"
              | "MOVING_AVG" "(" "window=" uint ")"
              | "CUMULATIVE" "(" "FROM" date_expr ")"
              | "GRAPH_SUM" "(" "edge_type=" string_lit "," "attr=" attr_key ")"

-- FROM clause
source     ::= cube_ref | grid_ref | "SHADOW CUBES" | "(" query ")"
cube_ref   ::= cube_id_literal | cube_name_string | namespace_path_string
grid_ref   ::= "GRID(" string_lit ")"

-- WHERE / predicates
where      ::= "WHERE" condition
condition  ::= predicate
              | "(" condition "AND" condition ")"
              | "(" condition "OR" condition ")"
              | "NOT" condition

predicate  ::= dim_pred | attr_pred | graph_pred | shadow_pred
dim_pred   ::= "D" uint "=" dim_key
              | "D" uint "IN" "[" dim_key ("," dim_key)* "]"
              | "D" uint "BETWEEN" dim_key "AND" dim_key
              | "D" uint "UNDER" dim_key          -- HierarchyAxis: node + all

descendants  | "D" uint "IS NULL"
            | "D" uint "IS NOT NULL"

attr_pred  ::= attr_key op val
              | attr_key "IS EMPTY" | "IS NOT EMPTY"
              | attr_key "INCLUDES" val          -- OR-Set contains element
              | attr_key "CONTAINS" val          -- text/json substring
              | attr_key "STARTS WITH" string_lit
              | attr_key "MATCHES" regex_lit
              | attr_key "HAS_SEVERITY" severity_lit

graph_pred ::= "CONNECTED_TO(" node_id_lit "," edge_type_lit ")"
              | "GRAPH_DISTANCE(" node_id_lit ")" op uint
              | "IN_SPACE(" space_id_lit ")"
```

```

shadow_pred ::= "IS SHADOW" | "IS NOT SHADOW"

-- Aggregation & pivot
fold        ::= "FOLD" axis_id "WITH" agg_fn "(" attr_key ")"
expand      ::= "EXPAND" axis_id "AS COLUMNS(" sel_item ")"
group       ::= "GROUP BY" group_item ("," group_item)* rollup?
group_item  ::= attr_key | "D" uint | alias
rollup      ::= ", ROLLUP(" axis_id ")"
order       ::= "ORDER BY" order_item ("," order_item)*
order_item  ::= (attr_key | "D" uint "." identifier | agg_expr)
              ("ASC" | "DESC")? ("NULLS FIRST" | "NULLS LAST")?
limit       ::= "LIMIT" uint ("OFFSET" uint)?

-- Query-level options
options     ::= option+
option      ::= "AS_OF" timestamp_expr           -- time-travel query
              | "IDENTITY" string_lit             -- scope to identity
partition   | "IN SPACE" string_lit               -- scope to Space
              | "INCLUDE SHADOW CELLS"            -- include all cross-grid
shadows     | "INCLUDE SHADOW FROM" string_lit     -- shadows from specific grid
only        | "TRAVERSE GRAPH" traversal_spec     -- graph traversal sub-plan
              | "WITH GRAPH OVERLAY"              -- return graph edges
alongside rows

traversal_spec ::=
  "FROM" (node_id_lit | "(" query ")")
  ("EDGE_TYPE" "=" edge_type_lit)?
  ("DIRECTION" "=" ("Outbound" | "Inbound" | "Both"))?
  ("MAX_DEPTH" "=" uint)?
  ("MIN_DEPTH" "=" uint)?
  ("EDGE_FILTER" "=" condition)?
  "SELECT" sel

```

4.2 Query Planner — Execution Pipeline

The HyperQL planner transforms a parsed AST into an optimized physical execution plan. It selects index strategies, pushes down VisibilityMask predicates early, and routes sub-plans to the appropriate backend (PostgreSQL for $N \leq 4$, ClickHouse for $N > 4$ aggregations).

```

impl HyperQLPlanner {
  pub fn plan(&self, query: &ParsedQuery) -> PhysicalPlan {
    // Phase 1: Semantic analysis
    // - Bind cube/attr/axis names to their IDs
    // - Validate axis references against Hypercube.dims
    // - Infer TypedAttrValue types for formula expressions
    let resolved = self.resolve_names(query);

    // Phase 2: Visibility pushdown (ALWAYS FIRST - security guarantee)
    // - Inject VisibilityMask DimSlice predicates into WHERE clause
    // - Add PermissionTier checks for any write paths
    // - Add TenantPartition / identity_tag filters for multi-identity queries
  }
}

```

```

// - This must happen before any index selection or optimization
let secured = self.apply_visibility_mask(&resolved, self.caller_identity);

// Phase 3: Predicate pushdown
// - Move DimSlice predicates as close to CellStore as possible
// - Estimate selectivity per predicate using CubeStatistics
// - Reorder predicates: most selective first
let pushed = self.push_predicates(&secured);

// Phase 4: Index selection per predicate
// - D1 EntityAxis equality → Hash index (primary key lookup)
// - D2 PropertyAxis equality/in-list → GIN index on JSONB attr_map
// - D3 TimeAxis range → BRIN index (chronological, low cardinality write)
// - D4 GeoAxis → PostGIS spatial index (bounding box or radius)
// - Arbitrary DimCoordinate → composite BTree index on dim key columns
// - N>4 FOLD aggregation + cardinality > threshold → route to ClickHouse
let indexed = self.select_indexes(&pushed);

// Phase 5: Join planning
// - CrossGridLink JOINS → plan shadow cell fetches (async, with consent
check) // - TRAVERSE GRAPH → BFS/DFS physical plan with depth limit and edge type
filter // - AS_OF → EventLog replay sub-plan targeting snapshot + delta window
let joined = self.plan_joins(&indexed);

// Phase 6: Aggregation planning
// - FOLD □ WITH AGG → GROUP BY + aggregation on fold axis keys
// - EXPAND □ AS COLUMNS → PIVOT sub-plan
// - ROLLUP → recursive hierarchical aggregation using closure table
// - MOVING_AVG / CUMULATIVE → window function plan on TimeAxis
let aggregated = self.plan_aggregations(&joined);

PhysicalPlan {
    steps: aggregated,
    estimated_rows: self.estimate_output_cardinality(&aggregated),
    backend: self.select_backend(&aggregated), //
Postgres|ClickHouse|Hybrid
}
}
}

```

4.3 AS_OF Time Travel — EventLog Replay

AS_OF reconstructs cell state at a specified historical timestamp. The implementation uses a snapshot-plus-delta approach: find the most recent snapshot before the target time, then replay EventLog entries up to the target. Direct full-history replay is only used when no snapshot is available.

```

impl TimeTraverser {
    pub fn get_cell_as_of(
        &self,
        coord: &DimCoordinate,
        as_of: DateTime<Utc>,
    )

```

```

) -> Option<HyperCell> {
  // Step 1: Find most recent snapshot before as_of
  let snapshot = self.snapshot_store
    .latest_before(coord.cube_id, as_of)
    .unwrap_or_else(|| Snapshot::genesis(coord.cube_id));
  // Genesis snapshot: empty cube at t=0. Forces full EventLog replay.

  // Step 2: Get cell from snapshot (may be None if it didn't exist yet)
  let mut cell = snapshot.get_cell(coord)
    .unwrap_or_else(|| HyperCell::empty(coord.clone()));

  // Step 3: Replay EventLog from snapshot.checkpoint_ts to as_of
  let delta = self.event_log.entries_for_coord(
    coord,
    snapshot.checkpoint_ts,
    as_of,
  );

  // Step 4: Apply each entry in causal order using merge engine
  // replay_mode=true: no new EventLog entries written, no conflict
surfacing
  for entry in delta {
    let op = CrdtOperation::from_event_entry(&entry);
    self.merge_engine.apply_replay(&op, &mut cell);
  }

  if cell.is_empty() { None } else { Some(cell) }
}

// Snapshot scheduling: called by background maintenance job
pub fn should_create_snapshot(
  &self,
  cube_id: CubeId,
  last_snapshot_ts: DateTime<Utc>,
  events_since_snapshot: u64,
) -> bool {
  let elapsed = Utc::now() - last_snapshot_ts;
  let stats = self.cube_stats.get(cube_id);
  let mutation_rate = stats.mutations_per_day;

  // Create snapshot if:
  // - Hot cube (>10K mutations/day): every 4 hours or every 50K events
  // - Normal cube: every 24 hours or every 10K events
  // - Cold cube: every 7 days or every 1K events
  match mutation_rate {
    r if r > 10_000.0 =>
      elapsed.num_hours() >= 4 || events_since_snapshot >= 50_000,
    r if r > 100.0 =>
      elapsed.num_hours() >= 24 || events_since_snapshot >= 10_000,
    _ =>
      elapsed.num_days() >= 7 || events_since_snapshot >= 1_000,
  }
}
}

```

4.4 DimFold and DimExpand — Physical Implementation

DimFold and DimExpand are the N-dimensional generalizations of GROUP BY and PIVOT. They are the backbone of Hypergrid's analytical capabilities — every dashboard, report, and KPI visualization in Kogi, Ume, and Qala uses one or both.

```
// DimFold: collapse one axis by aggregating all its key values into a scalar
// Example: FOLD D: WITH SUM(value) collapses the TimeAxis,
//           producing one row per entity with value = sum across all periods

// SQL translation for PostgreSQL backend:
// SELECT d1_key, SUM((attr_map->>'value')::numeric) AS folded_value
// FROM hypergrid_cells
// WHERE cube_id = $1
//       AND d2_key = 'revenue'      -- PropertyAxis filter
// GROUP BY d1_key;

// DimExpand: pivot one axis's key set into separate output columns
// Example: EXPAND D: AS COLUMNS(AVG(value))
//           turns CategoryAxis keys ["security","quality","compliance"]
//           into columns: security_avg, quality_avg, compliance_avg

// SQL translation (PostgreSQL crosstab via pivot):
// SELECT d1_key,
//       AVG(CASE WHEN d4_key = 'security' THEN (attr_map->>'value')::float
// END) AS security_avg,
//       AVG(CASE WHEN d4_key = 'quality' THEN (attr_map->>'value')::float
// END) AS quality_avg,
//       AVG(CASE WHEN d4_key = 'compliance' THEN (attr_map->>'value')::float
// END) AS compliance_avg
// FROM hypergrid_cells
// WHERE cube_id = $1
//       AND d2_key = 'score'
//       AND d3_key >= '2026-Q1'
// GROUP BY d1_key;

// N-Dimensional aggregation function set
pub enum AggFn {
    Sum, Avg, Count, CountUnique, Min, Max,
    Median, Percentile(f64), StdDev, Variance,
    Distribution, // enum frequency counts
    FoldSum, FoldAvg, // DimFold + aggregation
    Rollup { attr: AttributeKey }, // HierarchyAxis bottom-up
    Drilldown { attr: AttributeKey }, // HierarchyAxis top-down
    MovingAvg { window: u32 }, // TimeAxis window
    Cumulative { from: DateTime<Utc> }, // TimeAxis prefix sum
    GraphSum { edge_type: EdgeType, attr: AttributeKey }, // graph-connected sum
}
```

Chapter 5 — HG-COMP: Computation Engine

5.1 Tier-1 Formula Engine — Complete Function Library

The Tier-1 formula engine evaluates ComputedAttribute formulas synchronously on every read. It uses an interpreted AST evaluator with strict resource limits. The complete function library is listed below — all functions are available to all domain systems.

Category	Functions	N-Dim Extensions	Return Type
Arithmetic	+ − × ÷ % ^ ABS ROUND(x,n) CEIL FLOOR SIGN SQRT POW LN LOG EXP	Operands may be any DimCoordinate reference: CELL(D1=id, D2='revenue', D3='2026-Q2')	Number
Comparison	= != > < >= <= BETWEEN NOT_BETWEEN	Cross-dim: CELL(D3='2026-Q2', D2=attr) > CELL(D3='2026-Q1', D2=attr)	Bool
Logical	AND OR NOT IF(cond,t,f) IFS(c1,v1,...) SWITCH COALESCE ISNULL IFNULL NULLIF	Condition may reference any DimSlice predicate as a boolean expression	Dynamic
String	CONCAT UPPER LOWER LEN TRIM LEFT RIGHT SUBSTR CONTAINS STARTS ENDS REPLACE SPLIT REGEXMATCH FORMAT		String
Date/Time	TODAY NOW YEAR MONTH QUARTER WEEK DAY DAYS_BETWEEN DATE_ADD DATE_FORMAT FISCAL_QTR WORKDAYS_BETWEEN	Works naturally on TimeAxis keys: DATE_FORMAT(D3_KEY, 'YYYY-QQ')	Date/Number/String
Aggregation	SUM AVG COUNT COUNT_UNIQUE MIN MAX MEDIAN PERCENTILE(p) STDDEV VARIANCE DISTRIBUTION	Over SLICE: SUM(SLICE(D3='2026', D4='EMEA', D2='revenue'))	Number
Fold/Pivot	FOLD(axis_id, agg_fn, attr) FOLD_LAST_N(axis_id, n, agg_fn, attr)	Collapse any axis: FOLD(time_axis, SUM, 'revenue'); FOLD_LAST_N(time_axis, 4, AVG, 'score')	Number
Slice	SLICE(dim_predicates...) → cell_set	Any combination: SLICE(D1=active_entities, D3 >= ytd_start, D4 IN ['EMEA','APAC'])	Cell set

Category	Functions	N-Dim Extensions	Return Type
Graph	GRAPH_NEIGHBORS(edge_type, attr) GRAPH_PATH(to_id) GRAPH_DIST(to_id) GRAPH_CENTRALITY()	Traverses HG-GRAPH: GRAPH_NEIGHBORS('DependsOn', 'budget_allocated')	Number/List
Shadow/Cross Grid	SHADOW_CELL(grid_id, coord, attr) JOIN_GRID(grid_ref, d1_key, attr)	Reference a MirrorAttribute from a shadow cell in a linked Grid	Any
AI Engine	AI_SCORE(model_id, attrs...) AI_CLASSIFY(...) AI_PREDICT(...) AI_ANOMALY(...)	Injects Tier-2 AI signals as formula operands. Served from computed cache with TTL.	Number/String
Namespace	NS_RESOLVE(path) NS_PATH(entity_ref) NS_SCOPE(entity_ref) IS_UNDER_NS(path)		String/Bool
Space	SPACE_NAME() SPACE_TYPE() SPACE_MEMBER_COUNT() IN_SPACE(id)		String/Bool/Number
Lookup	RELATED(relation_attr, target_attr) LOOKUP(key, cube_ref, key_attr, value_attr) CORELATED(rel, filter, attr)	Cross-cube join: RELATED(customer_id, 'customer_name') from CRM cube	Dynamic
Identity	IDENTITY_TAG() IS_PARTITION(tag) VISIBLE_TO(observer_type)	VisibilityMask-aware: VISIBLE_TO('Public') returns bool	String/Bool
Time Travel	AS_OF(timestamp_expr, formula_expr)	Evaluates formula against historical cell state: AS_OF('2026-01-01', CELL(D1, 'status'))	Any
Link Network	LINK_COUNT() INBOUND_LINKS() OUTBOUND_LINKS() IS_LINKED_TO(entity_id) LINK_DEPTH()		Number/Bool
Engine Signals	HEALTH_SCORE() RISK_SCORE() MATCH_SCORE() INCOME_PROJECTION(days) SPACE_HEALTH()	Injects Tier-2 cached engine values as synchronous formula operands	Number

Formula evaluation resource limits (enforced by FormulaEvalLimits):

- Max distinct cell references per formula: 10,000
- Max rows in a SLICE aggregation: 1,000,000
- Max graph traversal depth via GRAPH_NEIGHBORS: 5
- Wall-clock execution limit: 100ms (Tier-1 formulas are inline in request path)
- Memory limit per formula evaluation: 64MB

5.2 Tier-2 AI Engine — Async Signal Architecture

Tier-2 computed attributes are asynchronous AI/ML signals. They are triggered by EventLog mutations, computed by registered AIEngineAdapter plugins, cached in Redis, and written back to HyperCells via the WritebackService. The full architecture is event-driven and pull-through-cache.

```
// Complete AIEngineAdapter plugin interface
pub trait AIEngineAdapter: Send + Sync {
    fn engine_id(&self) -> &str;           // e.g. "kogi.health_score_engine.v2"
    fn name(&self) -> &str;
    fn version(&self) -> semver::Version;

    // Which attribute keys does this engine produce?
    fn output_attrs(&self) -> Vec<AttributeKeyDef>;
    // All output attrs MUST have write_permission = PermissionTier::System

    // Which attribute keys does this engine consume as inputs?
    fn input_attrs(&self) -> Vec<AttributeKey>;

    // Which EventLog event types should trigger recomputation?
    fn invalidation_triggers(&self) -> Vec<EventKind>;

    // Should a specific attribute change trigger recompute for a given entity?
    fn should_recompute(
        &self,
        changed_attr: &AttributeKey,
        old_value: &TypedAttrValue,
        new_value: &TypedAttrValue,
    ) -> bool;

    // Compute a batch of entities (called by AI compute worker pool)
    async fn compute_batch(
        &self,
        requests: Vec<ComputeRequest>,
        ctx: &ComputeContext,
    ) -> Result<Vec<ComputeResult>, AIError>;

    // Detect anomalies across a DimSlice
    async fn detect_anomalies(
        &self,
        slice: &DimSlice,
        attr: &AttributeKey,
        sensitivity: f32,
    ) -> Result<Vec<AnomalySignal>, AIError>;

    // Natural language → HyperQL translation
    async fn nl_to_query(
        &self,
        nl: &str,
        cube_context: &CubeContext,
    ) -> Result<HyperQLQuery, AIError>;

    // Cache TTL for computed outputs (per model)
    fn cache_ttl(&self) -> Duration;
}

pub struct ComputeRequest {
```

```

    pub entity_id: DimKey,          // Di key of the entity to compute for
    pub cube_id:   CubeId,
    pub input_attrs: HashMap<AttributeKey, TypedAttrValue>, // current cell
values
    pub history:   Vec<EventEntry>, // recent EventLog entries for this entity
    pub graph_ctx: Option<GraphContext>, // linked entities if engine needs
graph data
}

pub struct ComputeResult {
    pub entity_id:   DimKey,
    pub outputs:     Vec<(AttributeKey, TypedAttrValue)>,
    pub computed_at: DateTime<Utc>,
    pub model_version:String,
    pub confidence:   f64,
}

```

5.3 WritebackService — gRPC Protocol

The WritebackService is the sole channel by which AI engines write computed attribute values into HyperCells. It enforces PermissionTier::System, validates that the engine is authorized to write each target attribute, appends the write to the EventLog, invalidates Redis cache, and broadcasts the update to WebSocket subscribers.

```

// WritebackService gRPC interface definition (proto3)
service WritebackService {
    rpc WriteColumn(WriteColumnRequest) returns (WriteColumnResponse);
    rpc WriteBatch(WriteBatchRequest) returns (WriteBatchResponse);
    rpc WriteAnomalyFlags(WriteAnomalyRequest) returns (WriteAnomalyResponse);
    rpc WriteObaHint(WriteObaHintRequest) returns (WriteObaHintResponse);
}

message WriteColumnRequest {
    string    component_id  = 1; // Di entity key
    string    cube_id      = 2;
    string    attr_key      = 3; // must be registered as attr_type=AI
    Value     value         = 4; // google.protobuf.Value (typed per attr_def)
    float     confidence    = 5; // 0.0-1.0
    string    source_engine = 6; // engine_id e.g. "kogi.health_score_engine.v2"
    int64     computation_ts = 7; // nanosecond epoch of computation
    int64     ttl_seconds   = 8; // Redis CellCache TTL
    string    space_id      = 9; // optional Space scope
    string    identity_tag  = 10; // optional partition scope
    string    model_version = 11; // for audit trail
    float     staleness_at  = 12; // invalidation time for this signal
}

// WritebackService processing pipeline (internal Go service):
// 1. Validate source_engine has AIWriter permission for attr_key
//    (check
PluginRegistry.engine_permissions[source_engine].allows_write(attr_key))
// 2. Evaluate PolicyEngine for any blocking policies on this write path
// 3. Threshold check: if value crosses alert threshold,
//    dispatch notification via HG-OPS notification service

```

```
// 4. Write value to CellStore as SetAttr with actor=ActorId::ai(source_engine)
// 5. Update Redis CellCache: SET cell:{cube_id}:{entity_id}:{attr_key}
//    with TTL=ttl_seconds and metadata (confidence, engine, computed_at)
// 6. Append EventKind::AIColumnUpdated to component EventLog
//    (includes: entity_id, attr_key, old_value, new_value, engine, model_version,
confidence)
// 7. WebSocket broadcast CellUpdateEvent to all Workspace subscribers
//    viewing any sheet that renders this attr_key for this entity
```

Chapter 6 — HG-VIEW, HG-SPACE, HG-NS, HG-ID

6.1 HypercubeView — Complete Data Model

```
pub struct HypercubeView {
  pub view_id:      ViewId,
  pub cube_id:      CubeId,
  pub name:         String,
  pub namespace_path: NamespacePath,

  // — Axis remapping —————
  pub primary_axis:  AxisId,          // which axis maps to "rows" in 2D
  rendering
  pub secondary_axis: AxisId,          // which axis maps to "columns"
  // Any dimension can be primary or secondary — not just D1/D2
  // Example: TimeAxis as primary → time periods as rows, entities as columns

  // — N-dim configuration —————
  pub dim_slices:    Vec<DimSlicePredicate>, // filter on
  non-primary/secondary axes
  pub dim_folds:     Vec<DimFold>,          // axes collapsed by aggregation
  pub dim_expands:   Vec<DimExpand>,        // axes expanded as pivot columns

  // — Column/row schema —————
  pub attr_schema:   Vec<AttrColumnDef>,    // which attrs shown + display
  order
  pub pinned_attrs:  Vec<AttributeKey>,      // always-visible (left freeze
  pane)
  pub hidden_attrs:  Vec<AttributeKey>,      // hidden from this view
  pub col_widths:    HashMap<AttributeKey, u32>, // DXA

  // — Filtering / sorting / grouping —————
  pub filters:       Vec<DimSlicePredicate>,
  pub sorts:         Vec<ViewSort>,         // (attr_key, direction,
  nulls_position)
  pub groups:        Vec<ViewGroup>,        // (group_by_attr, aggregations)
  pub highlight_rules: Vec<HighlightRule>,  // (condition, color_hex, label)
  pub summary_row:   Option<SummaryRowConfig>,

  // — Rendering —————
  pub render_mode:   RenderMode,
  // Grid2D|Kanban|Agile|Gantt|Calendar|Timeline|HierarchyTree|
  // Treemap|NetworkGraph|LinkForest|PivotTable|HeatMap|Matrix|
  // N-DimExplorer|Custom(PluginId)

  // — Augmentation —————
  pub shadow_policy:  ShadowCellPolicy,      // Include|Exclude|ShadowOnly
  pub identity_filter: Option<IdentityTag>,
  pub space_filter:   Option<SpaceId>,
  pub graph_overlay:  bool,
  pub ai_overlay:     bool,

  // — Sharing —————
}
```

```
pub visibility:      ViewVisibility, //
Private|Shared|SpaceShared|Public|Template
pub created_by:      IdentityId,
pub metadata:        AttributeMap,
}
```

6.2 Axis Remapping — Any Dimension Can Be Rows or Columns

One of the most powerful features of HypercubeView is axis remapping: the user chooses which dimension maps to rows and which to columns. This enables radically different perspectives on the same cube without changing underlying data.

View Name	primary_axis (rows)	secondary_axis (cols)	Use Case
Standard	D ₁ (Entity)	D ₂ (Property)	Conventional spreadsheet: entities as rows, field names as columns
Transposed	D ₂ (Property)	D ₁ (Entity)	Properties as rows, entities as columns — useful for attribute comparison views
Time Series	D ₃ (Time)	D ₂ (Property)	Time periods as rows, properties as columns — history timeline view
Geographic	D ₄ (Region)	D ₂ (Property)	Regions as rows, properties as columns — geographic breakdown view
Scenario Compare	D ₅ (Scenario)	D ₁ (Entity)	Scenarios as rows, entities as columns — plan-vs-actual comparison
Cross-dim Matrix	D ₃ (Time)	D ₄ (Region)	Time × Geography matrix with D ₁ -aggregated values — heatmap ready
Entity × Time	D ₁ (Entity)	D ₃ (Time)	Entities as rows, time periods as columns — classic time-series pivot
Org × Scenario	D ₆ (OrgUnit)	D ₅ (Scenario)	Org units as rows, scenarios as columns — planning comparison matrix

6.3 Render Mode Catalog

Render Mode	Primary/Secondary Axes	Best For	Implementation
Grid2D	Any D ₁ × D ₂	General editing and viewing	Standard spreadsheet table. Inline formula evaluation. CRDT real-time updates via WebSocket.
PivotTable	Any two + fold/expand	Multi-dimensional summaries	DimFold + DimExpand applied. Cross-tabulation of two axes with aggregate values.

Render Mode	Primary/Secondary Axes	Best For	Implementation
HeatMap	Any two numeric axes	Pattern detection, anomaly identification	Color-coded matrix: value-to-color mapping. Threshold rules per AttrKeyDef.
Kanban	$D_1 + D_2/D_3$ (status axis)	Workflow and status-driven work	Swimlane columns = enum axis values. Cards = row entities with key attrs. WIP limit enforcement.
Agile	D_1 (tasks) + sprint dim	Software development sprints	Sprint-scoped Kanban + burndown overlay + velocity tracking. Story point aggregation.
Gantt	D_1 (entity) + D_3 (time)	Project planning and timelines	Horizontal bars from start to end date. Dependency edges as arrows. Critical path highlight.
Calendar	D_3 (time) + D_1 (entity)	Scheduling and deadline management	Day/week/month grid rendering by date attribute. Multi-entity overlap detection.
Timeline	D_1 (entity) + D_3 (time)	Roadmaps and phased rollouts	Swimlane rows per entity; bars across time axis. Milestone markers.
HierarchyTree	D_1 with HierarchyAxis	Org charts, product taxonomies	Collapsible tree. ROLLUP aggregation at each node. Drill-down via HierarchyAxis UNDER queries.
Treemap	D_1 hierarchy + D_2 (size)	Budget composition, portfolio breakdown	Hierarchical tiles sized by numeric attr; color by score attr. Click-to-drill.
NetworkGraph	D_1 + HG-GRAPH edges	Dependency and link network visualization	Force-directed: nodes = row entities; edges = Hypergraph edges. D3.js + WebGL for large graphs.
LinkForest	D_1 + CrossGridLink edges	Inter-Grid connection visualization	KLNK-style tree with identity-colored branches. Multi-root forest layout.
N-DimExplorer	User-selectable N axes	Deep exploratory N-dim analysis	Interactive browser: axis selection dropdowns; DimSlice sliders for all non-primary axes. Live DimFold/DimExpand.
Custom	Plugin-defined	Domain-specific visualizations	HypercubePlugin implementing RenderModePlugin: render(view, data) → RenderOutput.

6.4 Space Internal Structure

```
pub struct Space {
    pub space_id:      SpaceId,          // UUID; also  $D_1$  key in system.spaces
    cube
    pub space_type:    SpaceType,        //
    Personal|Team|Organization|Cooperative|
                                     //
    Collective|Community|Federation|Project|
}
```

```

//
Research|Event|Studio|Factory|Custom
  pub slug:          String,          // URL-safe; unique within namespace
  pub name:          String,
  pub description:    Option<RichText>,
  pub namespace_path: NamespacePath,  //
kogi://org/pamoja/team/engineering/
  pub grid_id:        GridId,
  pub visibility:      Visibility,     //
Private|Protected|Internal|Public|Unlisted
  pub status:         SpaceStatus,    //
Draft|Active|Paused|Restricted|Dissolving|
// Dissolved|Archived

// — Members —————
pub members:          Vec<SpaceMember>,
// SpaceMember { user_id, role, joined_at, contribution_weight, stake_pct,
//               active_workspace_ids }

// — Cubes —————
pub cubes:            Vec<CubeId>,     // cubes owned by this Space
pub shared_cubes:      Vec<CubeId>,     // cubes shared from other Spaces

// — Governance —————
pub governance_config: GovernanceConfig,
// { vote_threshold_pct, quorum_pct, proposal_types, treasury_multisig_config,
//   dissolution_threshold, distribution_formula }
pub treasury_ref:      Option<TreasuryRef>, // kogi-bank account ref

// — Graph linkage —————
pub link_node_id:      NodeId,          // this Space's node in HG-GRAPH
pub space_link_subgraph: Option<SpaceLinkSubgraph>,

// — KLNK subgraph (materialized hourly) —————
// SpaceLinkSubgraph { nodes, edges, external_edges,
//                     last_materialized, member_count }
pub workspaces:        Vec<WorkspaceId>,
pub rooms:            Vec<RoomId>,
pub feed_id:          Option<ComponentId>,
pub template_id:       Option<TemplateId>,
pub federation_links: Vec<FederationLinkId>,
pub policy_ids:        Vec<PolicyId>,
pub shared_sheet_ids:  Vec<SheetId>,
pub toolbox_ids:       Vec<ToolBoxId>,
pub metadata:          AttributeMap,
}

```

6.5 Workspace — Active Session Structure

```

pub struct Workspace {
  pub workspace_id:      WorkspaceId,
  pub workspace_type:    WorkspaceType, //
Personal|Shared|Project|Guest|Template|AI
  pub owner_id:          IdentityId,
  pub space_id:          SpaceId,
  pub grid_id:           GridId,
  pub namespace_path:    NamespacePath,

```

```

// — Active session state —————
pub open_cubes:      Vec<OpenCubeRef>,
// OpenCubeRef { cube_id, view_id, active_dim_slices, scroll_state,
active_cell_coord }
pub active_cube_id:  Option<CubeId>,
pub pinned_rows:     Vec<(CubeId, DimKey)>,
pub pinned_columns:  Vec<(CubeId, AttributeKey)>,
pub recent_cells:    Vec<(CubeId, DimCoordinate, DateTime<Utc>)>,

// — Workspace-level DimContext (applied across all open cubes) —
pub dim_context:     HashMap<AxisId, DimKey>,
// Example: { time_axis_id → "2026-Q2" }
// ALL open cubes automatically show the Q2 slice when this is set.
// This is the N-dimensional generalization of "freeze panes":
// freezes a dimension at a specific key across ALL views simultaneously.

// — Global workspace filters —————
pub global_filters:  Vec<DimSlicePredicate>,
// Applied across ALL open cubes in this workspace.
// Example: identity_tag = "@jordan-dev-identity" to show only that partition.

// — UI state —————
pub layout_config:   WorkspaceLayout,
pub active_board_mode: Option<RenderMode>,
pub graph_panel_open: bool,

// — AI assistant state —————
pub ai_assistant_state: AIAssistantState,
// { context_window: Vec<Message>, active_cube_context, pending_actions:
Vec<ObaAction> }

// — Session metadata —————
pub session:         Option<ActiveSession>,
// ActiveSession { started_at, last_active, connected_clients: Vec<ClientId> }
pub metadata:         AttributeMap,
}

```

6.6 Namespace System — NamespacePath Resolution

```

// Namespace URI scheme
// hypergrid://{grid_name}/{space_type}/{slug}/{entity_type}/{entity_slug}/...
// Examples:
//   hypergrid://acme/org/engineering/           -- Engineering Space root
//   hypergrid://acme/org/engineering/cube/projects/ -- Projects Hypercube
//   hypergrid://acme/personal/alice/             -- Alice's Personal Space
//   hypergrid://acme/team/platform/cube/roadmap/row/feature-42/ -- HyperRow
//   hypergrid://global/community/data-science/  -- Community Space
//
// System-reserved:
//   hypergrid://{grid}/system/spaces/            -- Spaces Hypercube
//   hypergrid://{grid}/system/workspaces/        -- Workspaces Hypercube
//   hypergrid://{grid}/system/identities/        -- Identity registry
//   hypergrid://{grid}/system/namespaces/        -- Namespace registry
//   hypergrid://{grid}/system/graph/             -- Hypergraph store
//   hypergrid://{grid}/system/eventlog/          -- Global EventLog

```



```

pub struct NamespaceRegistry {
    // Primary store: PostgreSQL namespace_registry table
    // Read cache: Redis ns_resolve:{path} = {component_id, entity_type} TTL 300s
    pub store: Arc<dyn NamespaceStore>,
    pub cache: Arc<RedisCache>,
}

impl NamespaceRegistry {
    pub async fn resolve(&self, path: &NamespacePath) -> Option<EntityRef> {
        // 1. Check Redis cache (TTL 300s)
        if let Some(cached) = self.cache.get_ns(path).await { return Some(cached); }

        // 2. Query PostgreSQL namespace_registry WHERE path = $1
        let entry = self.store.get_by_path(path).await?;
        // 3. Populate Redis cache
        self.cache.set_ns(path, &entry.entity_ref, 300).await;
        Some(entry.entity_ref)
    }

    pub async fn register(&self, path: NamespacePath, entity_ref: EntityRef) ->
    NamespaceId {
        let entry = NamespaceEntry {
            namespace_id: Uuid::new_v4(),
            path: path.clone(),
            entity_ref,
            parent_id: self.resolve_parent(&path).await,
            ..Default::default()
        };
        self.store.insert(entry).await;
        self.cache.invalidate_prefix(&path.parent_path()).await;
        entry.namespace_id
    }

    pub async fn rename(&self, old_path: &NamespacePath, new_path: NamespacePath)
    {
        self.store.rename(old_path, new_path.clone()).await;
        // Create alias from old → new so existing references continue to work
        self.store.create_alias(old_path.clone(), new_path.clone()).await;
        // Async cascade: update child namespace entries
        self.cascade_rename_children(old_path, &new_path).await;
        self.cache.invalidate_prefix(old_path).await;
    }
}

```

6.7 Multi-Identity System — Complete Model

The multi-identity model allows one SovereignEntity to maintain multiple @handle Identities with independent partition visibility, namespace paths, and KLNK graph nodes. All identities share the same underlying HyperCells — they are logical partitions, not physical copies.

```

// Identity hierarchy:
// SovereignEntity → Account(s) → Identity(@handle) → ProfilePartition →
// VisibilityMask

```

```

pub struct ProfilePartition {
  pub partition_id:      Uuid,
  pub identity_handle:   String,      // @handle - unique within
  SovereignEntity
  pub sovereign_entity_id: EntityId,
  pub display_name:      String,
  pub isolation_level:    IsolationLevel,
  // IsolationLevel:
  //   None:      cross-identity reads permitted for authenticated owner
  //   SoftIsolated: cross-identity reads LOGGED as CrossPartitionAccessEvent
  //   HardIsolated: cross-identity reads BLOCKED at Substrate layer
  pub linked_accounts:   Vec<AccountId>,
  pub linked_spaces:     Vec<SpaceId>,
  pub visibility_mask:   VisibilityMask,
  pub public_view_id:    ViewId,
  pub follower_view_id:  ViewId,
  pub connection_view_id: ViewId,
  pub link_network_node_id: NodeId,  // KLNK node for this identity
  pub namespace_path:    NamespacePath, // kogi://user/{handle}/
  pub split_policy_id:   Option<PolicyId>,
}

pub struct VisibilityMask {
  pub public_rows:      RowFilter,
  pub follower_rows:    RowFilter,
  pub connection_rows:  RowFilter,
  pub public_attrs:     Vec<AttributeKey>, // visible to unauthenticated
  pub follower_attrs:   Vec<AttributeKey>, // visible to followers
  pub connection_attrs: Vec<AttributeKey>, // visible to trusted connections
  pub owner_only_attrs: Vec<AttributeKey>, // NEVER visible to non-owners
  pub custom_rules:     Vec<VisibilityRule>, // (DimSlicePredicate → attr_set)
  // Custom rules enable complex scenarios:
  // "WHEN D3 >= 'public_from_date' THEN visibility = Public" (time-scoped)
  // "WHEN D4 IN {permitted_regions} THEN visibility = Tenant" (geo-scoped)
  // "WHEN D1 IN GRAPH_NEIGHBORS('Collaborates') THEN visibility = Connection"
  // (network-scoped)
}

// N-dimensional visibility: VisibilityMask predicates are full DimSlice
// expressions.
// This enables visibility rules that vary by ANY combination of dimension axes:
//
// Row-level (conventional): D1 = {owned_by_me} OR D1.visibility = Public
// Column-level:             D2 IN {public_attrs}
// Time-scoped:              D3 >= {public_from_date}
// Geo-scoped:               D4 IN {permitted_regions}
// Scenario-scoped:          D5 IN {"base_case"} (non-owners see only base case)
// N-dim combined:           D1=active AND D3>=2026-Q1 AND D4 IN
// {allowed_regions}

```

6.8 CrossIdentityMerge Protocol

Step	Operation	Data Effect	Reversibile?
1. Pre-flight Check	Validate both source and target partitions belong to the same	No data change	N/A

Step	Operation	Data Effect	Reversible?
	SovereignEntity. Check for identity_tag conflicts.		
2. Checkpoint	Create a pre-merge snapshot of both partitions' HyperRow sets. Written to S3 with merge_checkpoint tag.	Snapshot written to S3	Yes — full restore
3. Tag Re-assignment	For rows tagged ONLY to source partition: OR-Set remove source_tag, OR-Set add target_tag.	identity_tags OR-Set ops on affected rows	Partial — checkpoint restore
4. Shared Tag Handling	For rows tagged to BOTH partitions: remove source_tag only. Rows remain in target partition.	identity_tags OR-Set remove for source_tag	Partial
5. Namespace Merge	Create namespace alias from source @handle → target @handle. Old @handle redirects.	NamespaceRegistry alias created	Yes — alias removable
6. KLNK Node Merge	Merge source identity's KLNK LinkNode into target. All source edges re-pointed to target node.	LinkEdge updates; source LinkNode archived	No — permanent
7. Space Membership	Transfer source identity's Space memberships to target identity.	SpaceMember records updated	Yes
8. Account Update	Re-associate accounts linked to source partition with target partition.	ProfilePartition.linked_accounts updated	Yes

Chapter 7 — HG-PLUGIN: Extension Interface

7.1 Complete Plugin Interface

All Hypergrid customizations — domain attribute sets, custom axis types, AI engines, render modes, governance models, and data connectors — are implemented as HypercubePlugin trait implementations. Plugins are loaded at Grid startup via dynamic linking or static registration and can be updated without downtime using hot-swap.

```
// Base HypercubePlugin trait — all plugins implement this
pub trait HypercubePlugin: Send + Sync {
    fn plugin_id(&self)      -> PluginId;
    fn plugin_name(&self)    -> &str;
    fn version(&self)        -> semver::Version;
    fn capabilities(&self)   -> Vec<PluginCapability>;
    // Capabilities: AxisType|AttributeType|ComputedModel|RenderMode|
    //               DataConnector|AIEngine|GovernanceModel|ExportFormat

    // Lifecycle hooks
    fn on_load(&mut self, ctx: &PluginContext) -> Result<(), PluginError>;
    fn on_unload(&mut self)                   -> Result<(), PluginError>;
    fn health_check(&self)                     -> PluginHealth;
    // PluginHealth: Healthy | Degraded { reason } | Unhealthy { reason }
}

// — AxisTypePlugin —————
pub trait AxisTypePlugin: HypercubePlugin {
    fn axis_type_id(&self) -> AxisType;
    fn validate_key(&self, key: &DimKey) -> Result<(), ValidationError>;
    fn compare_keys(&self, a: &DimKey, b: &DimKey) -> Ordering;
    fn key_to_display(&self, key: &DimKey) -> String;
    fn index_strategy(&self) -> IndexStrategy;
}

// — AttributeTypePlugin —————
pub trait AttributeTypePlugin: HypercubePlugin {
    fn attr_type_id(&self) -> AttributeType;
    fn validate_value(&self, v: &TypedAttrValue)
        -> Result<(), ValidationError>;
    fn merge(&self, a: &TypedAttrValue, b: &TypedAttrValue, ctx: &MergeContext)
        -> TypedAttrValue;
    fn render_cell(&self, v: &TypedAttrValue) -> RenderCell;
}

// — ComputedModelPlugin —————
pub trait ComputedModelPlugin: HypercubePlugin {
    fn model_id(&self) -> ModelId;
    fn output_attrs(&self) -> Vec<AttributeKeyDef>;
    fn compute(&self, row: &HyperRow, ctx: &ComputeContext)
        -> ComputeResult;
    fn invalidation_triggers(&self) -> Vec<EventKind>;
    fn cache_ttl(&self) -> Duration;
}
```

```
// — RenderModePlugin —————
pub trait RenderModePlugin: HypercubePlugin {
  fn render_mode_id(&self) -> RenderMode;
  fn render(&self, view: &HypercubeView, data: &ProjectedCube)
    -> RenderOutput;
  fn supported_axes(&self) -> Vec<AxisType>;
}

// — DataConnectorPlugin —————
pub trait DataConnectorPlugin: HypercubePlugin {
  fn connector_type(&self) -> ConnectorType;
  fn import(&self, source: ConnectorSource) -> Result<ImportPlan,
ConnectorError>;
  fn export(&self, cube: &HypercubeSlice, target: ConnectorTarget)
    -> Result<(), ConnectorError>;
  fn sync_stream(&self) -> Option<Box<dyn SyncStream>>;
}

// — AIEnginePlugin —————
pub trait AIEnginePlugin: HypercubePlugin {
  fn ai_engine_id(&self) -> AIEngineId;
  fn register_model(&self, model: AIModelDef) -> ModelId;
  fn compute_batch(&self, reqs: Vec<ComputeRequest>) -> Vec<ComputeResult>;
  fn detect_anomalies(&self, slice: &DimSlice, attr: &AttributeKey)
    -> Vec<AnomalySignal>;
  fn nl_to_query(&self, nl: &str, ctx: &CubeContext) -> HyperQLQuery;
}

// — GovernancePlugin —————
pub trait GovernancePlugin: HypercubePlugin {
  fn governance_model_id(&self) -> GovernanceModelId;
  fn evaluate_proposal(&self, proposal: &Proposal) -> GovernanceDecision;
  fn quorum_reached(&self, votes: &[Vote], total_members: u32) -> bool;
  fn compute_distribution(&self, contributors: &[Contributor]) ->
Vec<Distribution>;
}

// — EdgeTypePlugin —————
pub trait EdgeTypePlugin: HypercubePlugin {
  fn edge_type_id(&self) -> EdgeType;
  fn requires_consent(&self) -> bool;
  fn creates_shadow(&self) -> bool;
  fn validate_edge(&self, from: &HypergraphNode, to: &HypergraphNode)
    -> Result<(), ValidationError>;
  fn on_edge_accepted(&self, edge: &HypergraphEdge, grid: &mut Grid)
    -> Result<(), PluginError>;
}
```

7.2 Reference Plugin Catalog

Plugin Name	Type	Description
time-axis	AxisType	Built-in TimeAxis: Timestamp keys, BRIN index, chronological ordering, rollup by period (hour/day/week/month/quarter/year/fiscal), AS_OF time-travel support.

Plugin Name	Type	Description
geo-axis	AxisType	Built-in GeoAxis: GeoHash keys, PostGIS index, bounding box and radius queries, NUTS/ISO admin level hierarchies. Drill-down: continent→country→region→city.
hierarchy-axis	AxisType	Built-in HierarchyAxis: HierarchyPath keys, parent-child validation, ROLLUP and DRILLDOWN computation via closure table, cycle-safe tree traversal.
version-axis	AxisType	Built-in VersionAxis: SemVer keys, fork/merge operations, diff between version keys, semantic ordering.
tenant-axis	AxisType	Built-in TenantAxis: TenantId keys, row-level security per key, single-cube multi-tenant isolation.
openai-adapter	AIEngine	Reference OpenAI-compatible AI engine: NL-to-HyperQL, anomaly detection, health scoring, attribute completion. Configurable: OpenAI, Azure OpenAI, Ollama, any OpenAI-compatible API.
pm-domain	ComputedModel	Project management: health_score, schedule_risk, budget_risk, predicted_completion_date, velocity, cycle_time.
finance-domain	ComputedModel	Finance: profit, roi_pct, ytd_actuals, budget_variance, cash_flow_forecast, burn_rate.
crm-domain	ComputedModel	CRM: lead_score, pipeline_value, churn_probability, engagement_health, next_best_action.
kanban-renderer	RenderMode	Kanban board: swimlane columns = enum axis values; cards = row entities. WIP limits, drag-and-drop CRDT updates, custom card templates.
gantt-renderer	RenderMode	Gantt chart: horizontal bars on TimeAxis. Dependency edges as arrows. Critical path highlighting. Milestone markers. Baseline comparison.
ndim-explorer	RenderMode	N-Dim interactive explorer: axis selection dropdowns; DimSlice sliders for non-primary axes; real-time DimFold/DimExpand; graph overlay toggle.
network-graph-renderer	RenderMode	Force-directed graph: nodes = row entities; edges = Hypergraph edges. D3.js + WebGL for large graphs. Community cluster coloring. Centrality heat.
csv-connector	DataConnector	CSV import/export: column-to-attr mapping wizard, type inference, batch row creation with progress tracking, streaming export with DimFold pre-applied.
parquet-connector	DataConnector	Apache Parquet export: columnar format, Snappy compression, schema embedding, partition columns = dim axis keys. For data lake pipelines.
graphql-connector	DataConnector	GraphQL API adapter: expose any Hypercube as a GraphQL schema; mutation subscriptions via WebSocket.
cooperative-governance	Governance	Cooperative democracy: 1-member-1-vote, Shapley value distribution, multi-sig treasury operations, supermajority dissolution vote.

Plugin Name	Type	Description
democracy-governance	Governance	Democratic governance: configurable quorum %, supermajority options, ranked-choice voting (STV), proposal types: standard budget constitution dissolution.
multisig-treasury	Governance	M-of-N treasury: multi-sig approval for financial ops, escrow support, time-locked distributions, on-chain execution hook.

Chapter 8 — Persistence Architecture and Physical Schemas

8.1 Storage Layer Inventory

Store	Technology	Purpose	Access Pattern	Backup Strategy
Primary cell store	PostgreSQL 16 + JSONB	OLTP HyperCell reads/writes for N≤4 cubes. Primary entity tables per domain system.	Read-heavy; primary key lookup + JSONB GIN index; monthly partitioned event_log	Continuous WAL streaming; daily pg_dump to S3; PITR enabled
Analytics backend	ClickHouse	High-throughput DimFold/DimExpand for N>4 cubes and analytics workloads >1M rows.	Append-only writes via Kafka delta stream; aggregate reads via HyperQL router	Continuous replication to S3 Parquet; daily backup
CRDT hot buffer	Redis Cluster	CrdtLog in-flight operations; computed attribute cache; WebSocket session state.	Sub-millisecond R/W; TTL-based expiry; flushed to PostgreSQL on TTL	Redis RDB snapshots every 15min to S3. Non-durable by design.
EventLog cold archive	S3 / MinIO	EventLog entries after 10K-event hot cap; Parquet exports; AI model artifacts.	Write: periodic flush; Read: on AS_OF restore beyond hot window	S3 versioning + cross-region replication. Lifecycle: IA after 90d.
Search index	Meilisearch / Typesense	Full-text + faceted search across all HyperRows. Sub-10ms query latency.	Read: search queries; Write: async on component create/update	Rebuilt daily from PostgreSQL. No incremental backup needed.
Graph backend (v1)	PostgreSQL (recursive CTE)	Hypergraph edges, adjacency index, Space link subgraphs for graphs < 1M nodes.	Read: BFS/DFS via recursive CTE; Write: on edge create/delete	Same as primary cell store
Graph backend (v2.5)	Neo4j	Hypergraph at platform scale (> 1M nodes). KLNK traversal, community detection.	Neo4j native graph queries: MATCH patterns, shortestPath, Louvain	Neo4j Enterprise backup + cross-region
Snapshot store	S3 / MinIO	HyperCube point-in-time snapshots for AS_OF time-travel and disaster recovery.	Write: scheduled + manual; Read: on restore, on AS_OF beyond hot window	S3 versioning; lifecycle policies per cube retention config
Feature store	Redis + PostgreSQL MV	kogi-engine / Ume AI / Qala AI engine feature vectors and materialized view cache.	Read: per-entity on sheet load; Write: engine writeback via WritebackService	PostgreSQL MV rebuilt hourly from EventLog

8.2 Core PostgreSQL Schema — Kogi Domain

```
-- Primary component table (Kogi domain)
CREATE TABLE components (
    id                UUID PRIMARY KEY,
    owner_entity_id   UUID NOT NULL,          -- partition key: hash-partitioned
    by this
    item_type         VARCHAR(64),            --
Portfolio|Program|Project|Task|Artifact|...
    container_type    VARCHAR(64),            --
Space|Workspace|Portfolio|Program|...
    status            VARCHAR(32) NOT NULL DEFAULT 'Draft',
    visibility         VARCHAR(32) NOT NULL DEFAULT 'Private',
    space_id          UUID REFERENCES components(id),
    namespace_path    VARCHAR(512),
    identity_tags     TEXT[] DEFAULT '{}',    -- GIN indexed; partition tags
    tags              TEXT[] DEFAULT '{}',    -- GIN indexed; user tags
    metadata          JSONB NOT NULL DEFAULT '{}', -- ComponentMetadata
    data              JSONB NOT NULL DEFAULT '{}', -- ComponentData (typed
payload)
    health_score      FLOAT4,                -- cached from kogi-engine
WritbackService
    risk_score        FLOAT4,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY HASH (owner_entity_id);
-- 32 partitions recommended for > 100M rows

CREATE INDEX idx_comp_owner      ON components (owner_entity_id, status,
updated_at DESC);
CREATE INDEX idx_comp_type      ON components (item_type, status, visibility);
CREATE INDEX idx_comp_space     ON components (space_id, status);
CREATE INDEX idx_comp_tags     ON components USING GIN (tags);
CREATE INDEX idx_comp_itags     ON components USING GIN (identity_tags);
CREATE INDEX idx_comp_ns       ON components (namespace_path
varchar_pattern_ops);
CREATE INDEX idx_comp_scores    ON components (health_score DESC, risk_score
DESC);
CREATE INDEX idx_comp_ns_path   ON components (namespace_path);

-- Graph edges (Hierarchy, Dependency, CrossGridLink, etc.)
CREATE TABLE graph_edges (
    edge_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_id         UUID NOT NULL,
    target_id         UUID NOT NULL,
    edge_type         VARCHAR(64) NOT NULL,
    direction         VARCHAR(16) NOT NULL DEFAULT 'Directed',
    weight            FLOAT4 DEFAULT 1.0,
    attributes        JSONB DEFAULT '{}',          -- edge-level AttributeMap
    consent_status    VARCHAR(32) DEFAULT 'None', --
None|Pending|Accepted|Declined|Revoked
    consent_config    JSONB,                      -- ConsentConfig if CrossGridLink
    space_id          UUID,                        -- NULL = global; non-NULL =
Space-scoped
    expires_at        TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    created_by        VARCHAR(256) NOT NULL
);
CREATE INDEX idx_edges_source ON graph_edges (source_id, edge_type,
consent_status);
CREATE INDEX idx_edges_target ON graph_edges (target_id, edge_type);
```

```

CREATE INDEX idx_edges_space ON graph_edges (space_id, edge_type) WHERE space_id
IS NOT NULL;

-- EventLog: append-only, monthly time-partitioned
CREATE TABLE event_log (
    entry_id        UUID DEFAULT gen_random_uuid(),
    component_id    UUID NOT NULL,
    actor_id        VARCHAR(256) NOT NULL,          --
    "user_id:{identity_tag}:{node_id}"
    event_type      VARCHAR(64) NOT NULL,          -- CrdtOperation type
    attr_key        VARCHAR(256),
    before_value    JSONB,
    after_value     JSONB,
    delta           JSONB NOT NULL,                -- full CrdtOperation payload
    vector_clock    JSONB NOT NULL,
    space_id        UUID,
    workspace_id    UUID,
    identity_tag    VARCHAR(256),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (entry_id, created_at)             -- composite for partition
) PARTITION BY RANGE (created_at);
-- Monthly partitions: event_log_2026_01, event_log_2026_02, etc.
-- PostgreSQL RLS policy: NO UPDATE, NO DELETE (append-only enforcement)

-- Shadow rows (CrossGridLink reflections)
CREATE TABLE shadow_rows (
    shadow_id        UUID PRIMARY KEY,
    source_grid_id   UUID NOT NULL,
    source_dl_key     UUID NOT NULL,
    source_edge_id   UUID NOT NULL REFERENCES graph_edges(edge_id),
    shadow_status    VARCHAR(32) DEFAULT 'Active', --
Active|Degraded|Disconnected
    visible_attrs    JSONB NOT NULL,              -- consented attribute keys
    write_back_attrs JSONB DEFAULT '[]',          -- write-back permitted attribute
keys
    update_policy    VARCHAR(32) DEFAULT 'RealTime',
    last_synced_at   TIMESTAMPTZ,
    sync_error       TEXT,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Namespace registry
CREATE TABLE namespace_registry (
    namespace_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    path             VARCHAR(512) UNIQUE NOT NULL,
    component_id     UUID NOT NULL REFERENCES components(id),
    entity_type      VARCHAR(64) NOT NULL,
    slug             VARCHAR(128) NOT NULL,
    parent_ns_id     UUID REFERENCES namespace_registry(namespace_id),
    visibility        VARCHAR(32) NOT NULL DEFAULT 'Public',
    aliases          TEXT[] DEFAULT '{}',
    canonical_url    VARCHAR(1024),
    federation_origin VARCHAR(256),              -- origin grid_id if federated entry
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_resolved_at TIMESTAMPTZ
);
CREATE INDEX idx_ns_path ON namespace_registry (path);
CREATE INDEX idx_ns_parent ON namespace_registry (parent_ns_id);
CREATE INDEX idx_ns_slug ON namespace_registry (slug, entity_type);

```

8.3 Hypergrid Core Cell Store Schema

```
-- Universal Cell Store: generic N-dimensional cell table
-- Used by Hypergrid core for cubes not handled by domain-specific tables
CREATE TABLE hypergrid_cells (
    grid_id          UUID NOT NULL,
    cube_id          UUID NOT NULL,
    dim1_key         UUID NOT NULL,          -- EntityAxis key (D1)
    dim2_key         TEXT NOT NULL,         -- PropertyAxis key (D2, field
name)
    dim3_key         TEXT,                  -- Optional: TimeAxis /
CategoryAxis
    dim4_key         TEXT,                  -- Optional: GeoAxis / TenantAxis
    dim5_key         TEXT,                  -- Optional: additional axis
    -- dim6..dim16_key omitted here; added via ALTER TABLE for higher-dim cubes
    attr_value       JSONB NOT NULL DEFAULT '{}', -- TypedAttrValue as JSONB
    attr_version     BIGINT NOT NULL DEFAULT 0,
    attr_crdt_ts     BIGINT NOT NULL,       -- logical clock timestamp for LWW
    last_actor       TEXT NOT NULL,        --
"{tenant_id}:{identity_tag}:{node_id}"
    last_mutated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (grid_id, cube_id, dim1_key, dim2_key,
                COALESCE(dim3_key, ''), COALESCE(dim4_key, ''),
                COALESCE(dim5_key, ''))
) PARTITION BY HASH(grid_id); -- horizontal sharding by grid

-- TimeAxis cells: BRIN index on dim3_key for chronological range queries
CREATE INDEX idx_cells_time ON hypergrid_cells USING BRIN (dim3_key)
    WHERE dim3_key IS NOT NULL;

-- PropertyAxis: GIN on attr_value JSONB for attribute predicate pushdown
CREATE INDEX idx_cells_attr ON hypergrid_cells USING GIN (attr_value);

-- CrdtLog: in-flight CRDT operation buffer (flushed to EventLog on commit)
CREATE TABLE hypergrid_crdt_log (
    op_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    grid_id         UUID NOT NULL,
    cube_id         UUID NOT NULL,
    op_type         TEXT NOT NULL,          -- CrdtOperation variant name
    op_payload       JSONB NOT NULL,        -- serialized CrdtOperation
    vector_clock     JSONB NOT NULL,
    actor           TEXT NOT NULL,
    applied         BOOLEAN NOT NULL DEFAULT FALSE,
    federated_to    TEXT[] DEFAULT '{}',    -- peer node_ids this op has been
sent to
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_crdt_unapplied ON hypergrid_crdt_log (grid_id, applied,
created_at)
    WHERE applied = FALSE;
```

Chapter 9 — Security, Authentication, and Authorization Pipeline

9.1 Request Authentication Pipeline

Every request to any Apapo service follows a strict sequential pipeline. No step can be bypassed. The pipeline is implemented as a Go middleware chain in the api-gateway service.

```
// Every request to any KIMDSS/Hypergrid service follows this pipeline:

// Step 1: api-gateway - token validation and extraction
//   - Validate JWT / API key / session token (RS256 JWT signature check)
//   - Extract from token: sovereign_entity_id, active_identity_tag,
//                         active_space_id, active_workspace_id, scopes
//   - Rate limit check: per-entity (100 req/s) + per-endpoint (configurable)
//   - CORS check for browser clients
//   - Request ID injection for distributed tracing

// Step 2: identity-service - partition resolution
//   - Resolve ProfilePartition for active_identity_tag
//   - Load VisibilityMask for this partition (Redis-cached, 60s TTL)
//   - Check IsolationLevel if cross-partition data access is requested:
//     - None: permit
//     - SoftIsolated: permit + log CrossPartitionAccessEvent
//     - HardIsolated: DENY + return 403

// Step 3: portfolio-service / space-service - PermissionTier check
//   - Evaluate ComponentData.users or SpaceMember.role for caller's tier
//   - Tiers: Viewer(1) Subscriber(2) Contributor(3) Editor(4) Manager(5) Owner(6) Admin(7)
//   - Short-circuit: if caller's tier < required tier for operation → DENY

// Step 4: PolicyEngine - policy evaluation
//   - Evaluate all policy_ids attached to target entity
//   - PolicyDecision: Allow | Deny(reason) | RequireApproval(reason, approvers)
//   - SplitPolicy: validate cross-partition write rules
//   - GovernancePolicy: check if mutation requires GovernanceProposal vote

// Step 5: Row-Level Security - PostgreSQL RLS
//   - Injected WHERE clause:
//     visibility IN (allowed_set) OR owner_entity_id = $caller_id
//   - identity_tags filter: identity_tags && ARRAY[visible_partitions]
//   - space_id filter: if space-scoped request, add space_id = $active_space_id

// Step 6: Column-Level Masking - application layer
//   - Apply VisibilityMask: strip owner_only attrs from non-owner responses
//   - ShadowRow enforcement: return only visible_columns for shadow rows
//   - AI-write-only attrs: readable but never writable by non-System actors

// Step 7: Audit logging
//   - Every authenticated request logged to HG-OPS access_log
//   - Mutations also logged to EventLog with full actor context
//   - Cross-partition access events logged to separate compliance_log
```

9.2 Complete Permission Matrix

Action	Viewer	Subscriber	Contributor	Editor	Manager	Owner	Admin
Read public components	✓	✓	✓	✓	✓	✓	✓
Read follower-visible components	✗	✓	✓	✓	✓	✓	✓
Read connection-visible components	✗	✗	✓	✓	✓	✓	✓
Follow / Subscribe / Watch	✗	✓	✓	✓	✓	✓	✓
Create KLNK link request	✗	✓	✓	✓	✓	✓	✓
Submit contribution	✗	✗	✓	✓	✓	✓	✓
Edit component fields	✗	✗	✗	✓	✓	✓	✓
Create child components	✗	✗	✗	✓	✓	✓	✓
Add custom columns / attr keys	✗	✗	✗	✓	✓	✓	✓
Create / manage Workspaces	✗	✗	✗	✓	✓	✓	✓
Manage Space settings	✗	✗	✗	✗	✓	✓	✓
Manage Space members	✗	✗	✗	✗	✓	✓	✓
Submit governance proposals	✗	✗	✗	✗	✓	✓	✓
Manage treasury	✗	✗	✗	✗	✓	✓	✓
Configure identity partitions	✗	✗	✗	✗	✗	✓	✓
Archive / delete components	✗	✗	✗	✗	✗	✓	✓
CrossIdentityMerge	✗	✗	✗	✗	✗	✓	✓
Dissolve Space	✗	✗	✗	✗	✗	✓	✓
Platform-level admin override	✗	✗	✗	✗	✗	✗	✓

9.3 Data Encryption Reference

Data Category	Method	Key Management	Notes
All data in transit	TLS 1.3	Platform-managed certs (Let's Encrypt / ACM)	All HTTP, WebSocket, gRPC services. Mutual TLS for internal gRPC service mesh.

Data Category	Method	Key Management	Notes
All data at rest (PostgreSQL)	AES-256 TDE	pgcrypto; keys in AWS KMS or HashiCorp Vault	Storage-level encryption. Transparent to application layer.
PII fields	Field-level AES-256	Per-SovereignEntity key derivation from master key in Vault	legal_name, phone, email, gov_id, bank_account_ref. Application-level encryption before write.
Health / benefit data	Field-level AES-256	Same as PII	BenefitAccount balances, claim details, HSA transactions.
HardIsolated partition rows	Extra field-level AES-256	Per-partition key derived from per-entity key	Rows tagged to HardIsolated partition encrypted with partition-specific key. Double-encrypted.
EventLog entries (S3 archive)	AES-256 SSE-S3	S3-managed or KMS-managed	HMAC-SHA256 integrity signatures. Tamper-evident chain for regulated industries.
Snapshot archives (S3)	AES-256 SSE-S3	S3-managed or KMS-managed	Cross-region replication. Lifecycle: Glacier after 90d for cold cubes.
GDPR export archives	AES-256 + user passphrase (optional)	Platform signing key + optional user passphrase	Recipient-verifiable integrity. Machine-readable JSON-LD with schema definitions.

Chapter 10 — Domain-Layer Bridge Pattern: DomainStore Implementation

10.1 Bridge Layer Architecture

Every domain system (Kogi, Ume, Qala) connects to the Hypergrid substrate through a Bridge Layer: a set of DomainStore implementations that translate domain entities to HyperCells and back. The Bridge Layer is the sole point of contact between domain logic and Hypergrid primitives. Domain entities are pure Rust structs with no storage concerns; the bridge handles all serialization.

```
// DomainStore trait — every domain entity type implements this
pub trait DomainStore<Entity, EntityId>: Send + Sync {
    /// Register all Hypercubes and attribute schemas on Grid startup.
    fn bootstrap(grid: &mut Grid) -> HypergridResult<CubeId>;

    /// Write a domain entity into the Hypercube as a set of HyperCells.
    fn write(
        grid: &mut Grid,
        cube_id: CubeId,
        entity: &Entity,
        actor: &str,
    ) -> HypergridResult<()>;

    /// Read all HyperCells for an EntityId and reconstruct the domain entity.
    fn read(
        grid: &Grid,
        cube_id: CubeId,
        id: EntityId,
    ) -> DomainResult<Option<Entity>>;

    /// Check whether an entity exists in the Hypercube.
    fn exists(
        grid: &Grid,
        cube_id: CubeId,
        id: EntityId,
    ) -> bool;

    /// Query multiple entities matching a predicate.
    fn query(
        grid: &Grid,
        cube_id: CubeId,
        slice: &DimSlice,
        sort: Option<ViewSort>,
        limit: u32,
        offset: u32,
    ) -> DomainResult<Vec<Entity>>;
}

// — Example: Kogi ComponentStore (Bridge Layer) —————
pub struct ComponentStore;
```

```

impl DomainStore<Component, ComponentId> for ComponentStore {
  fn bootstrap(grid: &mut Grid) -> HypergridResult<CubeId> {
    let cube_id = grid.register_hypercube(HypercubeSpec {
      name: "kogi.portfolio.components".into(),
      dims: vec![
        DimensionAxis { dim_index: 1, axis_type: AxisType::EntityAxis,
          key_type: KeyType::Uuid, ..Default::default() },
        DimensionAxis { dim_index: 2, axis_type: AxisType::PropertyAxis,
          key_type: KeyType::String, ..Default::default() },
      ],
      encoding: StorageEncoding::Hybrid { dense_dims: vec![1, 2] },
    })?;

    // Register all attribute keys with their CRDT semantics
    grid.register_attr_key(cube_id, lww_text("name", "Name"))?;
    grid.register_attr_key(cube_id, lww_text("description", "Description"))?;
    grid.register_attr_key(cube_id, or_set_tags("tags", "Tags"))?;
    grid.register_attr_key(cube_id, lattice_status("status", "Status",
component_lattice()))?;
    grid.register_attr_key(cube_id, lww_json("data", "Component Data"))?;
    grid.register_attr_key(cube_id, ai_number("health_score", "Health Score",
      "kogi.health_score_engine.v2", Duration::from_secs(1800)))?;
    grid.register_attr_key(cube_id, ai_number("risk_score", "Risk Score",
      "kogi.risk_engine.v2", Duration::from_secs(900)))?;
    grid.register_attr_key(cube_id, max_register_u64("version", "Version"))?;
    grid.register_attr_key(cube_id, lww_text("visibility", "Visibility"))?;
    grid.register_attr_key(cube_id, or_set_uuids("owners", "Owners"))?;
    grid.register_attr_key(cube_id, or_set_text("identity_tags", "Identity
Tags"))?;
    // ... additional attr keys ...

    Ok(cube_id)
  }

  fn write(grid: &mut Grid, cube_id: CubeId, entity: &Component, actor: &str)
    -> HypergridResult<()>
  {
    let dl = DimKey::Uuid(entity.metadata.id);
    let ops = vec![
      set_attr(dl, "name",          TextValue(&entity.data.name),
actor),
      set_attr(dl, "description",   TextValue(&entity.data.description),
actor),
      set_attr(dl, "status",        JsonValue(&entity.data.status),
actor),
      set_attr(dl, "visibility",    TextValue(&entity.data.visibility),
actor),
      set_attr(dl, "data",          JsonValue(&entity.data),
actor),
      set_attr(dl, "version",       IntValue(entity.metadata.version),
actor),
      // identity_tags: each tag is an OR-Set add with unique_tag = tag_hash
      // This ensures concurrent tag additions from multiple nodes both
survive
    ];
    for op in ops { grid.crdt_engine.apply_crdt_op(&op, &mut
*grid.cell_store.write(),
                                &mut
*grid.event_log.write())?; }
    Ok(())
  }
}

```



```

fn read(grid: &Grid, cube_id: CubeId, id: ComponentId) ->
DomainResult<Option<Component>> {
    let cells = grid.cell_store.scan_row(cube_id, &DimKey::Uuid(id));
    if cells.is_empty() { return Ok(None); }

    // Reconstruct domain entity from HyperCells
    let mut builder = ComponentBuilder::new(id);
    for cell in cells {
        let field = cell.coord.keys[1].as_str(); // D2 = field name
        let value = cell.attributes.get("value");
        match field {
            "name" => builder.name(value?.as_str()?),
            "description" => builder.description(value?.as_str()?),
            "status" => builder.status(value?.as_json()?),
            "data" => builder.data(value?.as_json()?),
            "health_score" => builder.health_score(value?.as_f64()),
            "risk_score" => builder.risk_score(value?.as_f64()),
            _ => {} // unknown attr keys: ignore (forward-compatible)
        }
    }
    Ok(Some(builder.build()?))
}

```

10.2 Qala Domain Attribute Reference — qala.sdes

D ₂ Key	TypedAttrValue	CRDT	Permission Tier	Notes
name	Text	LWW	Editor	SDE display name
factory_id	Json	LWW	Admin	Parent Solution Factory UUID
owner_id	Json	LWW	Admin	SDE owner identity
template_id	Json	LWW	Admin	Template used to provision this SDE
status	Json	Lattice	System	Provisioning → Active → Suspended → Decommissioned
deployment_target	Json	LWW	Editor	Platform Cloud Machine Edge Air-gapped
environment_manifest	Json	LWW	Editor	.qala.yaml full environment specification
toolchain_ids	Json	OR-Set	Editor	Registered toolchain UUIDs
solution_ids	Json	OR-Set	System	Solutions produced in this SDE
language_runtimes	Json	LWW	Editor	Supported language runtime configurations
hermetic_manifest	Json	LWW	Editor	Digest-pinned dependency manifest for hermetic builds
config_files	Json	LWW	Editor	Layered configuration: Factory > SDE > Solution
env_variables	Json	LWW	Admin	Environment variable set (encrypted reference, not plaintext)

D ₂ Key	TypedAttrValue	CRDT	Permission Tier	Notes
snapshot_ids	Json	OR-Set	System	Historical SDE snapshot IDs
drift_status	Json	LWW (AI-write)	System	DriftDetectionResult from DriftDetectionEngine. 15min SLA.
ai_recommendation_ids	Json	OR-Set	System	AI Agent recommendation UUIDs for this SDE. OR-Set so concurrent recs survive.
last_build_at	Json	LWW	System	Most recent build timestamp
last_snapshot_at	Json	LWW	System	Most recent snapshot timestamp
provisioning_time_ms	Number	MaxRegister	System	Time from request to Active state (best recorded, not averaged)
version	Text	MaxRegister	System	SDE configuration version — always the maximum seen

10.3 Qala Domain Attribute Reference — qala.solutions

D ₂ Key	TypedAttrValue	CRDT	Permission Tier	Notes
name	Text	LWW	Editor	Solution display name
solution_type	Json	LWW	Editor	Application System Good Product Service Platform
sde_id	Json	LWW	Editor	Parent SDE UUID
model_id	Json	LWW	Editor	Solution Model this instantiates
lifecycle_state	Json	Lattice	Manager	Draft→InReview→Approved→Active→Deprecated→Retired
version	Text	MaxRegister	System	SemVer solution version string
structure	Json	LWW	Editor	Seven-level decomposition (System→App→Process→Module→Component→Function→Data)
playbook_id	Json	LWW	Editor	Associated Playbook document ID
artifact_ids	Json	OR-Set	System	Build artifact UUIDs
ccr_ids	Json	OR-Set	Manager	Change Control Request UUIDs — OR-Set: concurrent CCR creation safe
release_ids	Json	OR-Set	Manager	Release record UUIDs
risk_score	Number	LWW (AI-write)	System	QalaRiskEngine output [0–100]. Confidence stored in AiSignal wrapper.
quality_score	Number	LWW (AI-write)	System	QalaQualityEngine output [0–100]

D ₂ Key	TypedAttrValue	CRDT	Permission Tier	Notes
security_score	Number	LWW (AI-write)	System	QalaSecurityEngine output [0–100]
compliance_status	Json	LWW (AI-write)	System	Domain-pack compliance check result (per governance pack)
domain_pack_data	Json	LWW	Editor	Domain Pack extension fields (pharma batch fields, financial KYC, etc.)
owner_id	Json	LWW	Admin	Primary solution owner identity
contributing_owners	Json	OR-Set	Manager	Contributing owner identity IDs

10.4 kogi-engine Sub-System Registry — Complete Reference

Engine	Scala Class	Output Columns	Cache TTL	Invalidation Trigger
AnalyticsEngine	AnalyticsEngine	views, clicks, ctr, engagement_rate, followers, sentiment_score	30 min	Any analytics event
TelemetryEngine	TelemetryEngine	event_count, mutation_velocity, last_event_at, flow_stats	5 min	Any EventLog entry
RecommendationEngine	RecommendationEngine	recommendation_score, similar_items, suggested_actions	1 hr	Significant portfolio composition change
PersonalizationEngine	PersonalizationEngine	personalized_rank, relevance_score, persona_label	2 hr	User preference or behavior change
MatchEngine	MatchEngine	match_score, talent_match_score, investor_match_score	2 hr	Skill, requirement, or availability change
GraphEngine	GraphEngine	dependency_depth, critical_path_flag, network centrality	1 hr	Graph edge create/delete
RiskEngine	RiskEngine	risk_score, risk_breakdown, mitigation_gap_flags, at_risk_deadline_count	15 min	risk_flag change; deadline approach (< 7 days)
OptimizationEngine	OptimizationEngine	predicted_completion_date, schedule_risk_flag, budget_risk_flag, resource_bottlenecks	30 min	Sprint completion; task update; allocation change
SearchEngine	SearchEngine	search_rank, full_text_index_status, keyword_suggestions	N/A	Component name/description/tag change
AllocationEngine	AllocationEngine	capacity_conflicts, allocation_recommendations, utilization_heatmap	15 min	Allocation record change

Engine	Scala Class	Output Columns	Cache TTL	Invalidation Trigger
IncentiveEngine	IncentiveEngine	kogipoints_earned, reputation_tier, badges, incentive_streak	1 hr	Action event (contribution, governance vote, etc.)
CollaborationEngine	CollaborationEngine	collaboration_score, contributor_diversity_index, merge_conflict_count, attribution_weights	30 min	Contribution event; CRDT conflict
BenefitsEngine	BenefitsEngine	benefits_health_score, coverage_gap_flags, tax_savings_estimate	6 hr	Benefit account or gig change
GrantEngine	GrantEngine	grant_match_score, eligibility_flags, recommended_grants	24 hr	Portfolio composition significant change
CrowdfundingEngine	CrowdfundingEngine	campaign_health_score, success_probability, investor_match_quality	5 min (active campaigns)	Contribution event; backer count change
SpaceHealthEngine	SpaceHealthEngine	space_health_score, at_risk_members, governance_health, treasury_status	1 hr	Space event (member change, governance action, etc.)
CRMEngine	CRMEngine	lead_score, pipeline_value, follow_up_due_flags, churn_probability	2 hr	Contact interaction event
NamespaceEngine	NamespaceEngine	namespace_health_score, stale_paths, broken_paths, resolve_latency_p99	1 hr	Namespace event (rename, alias, federation sync)

Chapter 11 — Complete API Surface Reference

11.1 Hypergrid Core REST API

Endpoint	Method	Description	Min Auth
/grids	GET	List all Grids accessible to the caller	Viewer
/grids	POST	Create a new Grid deployment	Admin
/grids/{id}/cubes	GET	List all Hypercubes in a Grid	Viewer
/cubes	POST	Create a new Hypercube with dimension axis config	Editor
/cubes/{id}	GET	Get Hypercube schema, dimensions, and metadata	Viewer
/cubes/{id}/dims	GET	List all DimensionAxes for a Hypercube	Viewer
/cubes/{id}/dims	POST	Add a new DimensionAxis (N+1) — zero-downtime schema evolution	Manager
/cubes/{id}/attrs	GET	List AttributeKeyRegistry for a Hypercube	Viewer
/cubes/{id}/attrs	POST	Register a new attribute key with CRDT semantics	Editor
/cubes/{id}/rows	GET	Query HyperRows (DimSlice params + pagination)	Viewer
/cubes/{id}/rows	POST	Create a new HyperRow (set D ₁ key + initial attribute map)	Editor
/cubes/{id}/rows/{d1_key}	GET	Get all cells for one HyperRow (all D ₂ attrs)	Viewer
/cubes/{id}/rows/{d1_key}	PATCH	Update cell attribute values for a HyperRow (CrdtOps batch)	Editor
/cubes/{id}/rows/{d1_key}	DELETE	Soft-archive a HyperRow (sets status=Archived)	Owner
/cubes/{id}/cells/{coord}	GET	Get a single HyperCell by N-dim coordinate tuple	Viewer
/cubes/{id}/cells/{coord}	PUT	Set a HyperCell attribute map at a coordinate	Editor
/cubes/{id}/cells/{coord}/{key}	PATCH	Update a single attribute of a HyperCell (one CrdtOp)	Editor
/cubes/{id}/query	POST	Execute a HyperQL query against this Hypercube	Viewer

Endpoint	Method	Description	Min Auth
/cubes/{id}/views	GET	List HypercubeViews for this cube	Viewer
/cubes/{id}/views	POST	Create a new HypercubeView	Editor
/cubes/{id}/views/{view_id}/rows	GET	Render rows for a specific view (all view config applied)	Viewer
/cubes/{id}/fold	POST	Execute a DimFold aggregation across a specified axis	Viewer
/cubes/{id}/expand	POST	Execute a DimExpand pivot across a specified axis	Viewer
/cubes/{id}/export	GET	Export cube or view (CSV/XLSX/JSON/Parquet/Arrow/JSON-LD)	Editor
/cubes/{id}/snapshot	POST	Create a named cube snapshot (point-in-time for AS_OF)	Owner
/cubes/{id}/restore/{snap_id}	POST	Restore cube from snapshot	Owner
/cubes/{id}/eventlog	GET	Get EventLog entries for this cube (paginated, AS_OF supported)	Editor
/cubes/{id}/crdt/sync	POST	Submit CrdtLog batch (federation peers only — System auth)	System

11.2 Hypergraph and Federation API

Endpoint	Method	Description	Min Auth
/graph/nodes/{id}/tree	GET	Get link tree from node (depth, edge_type, direction params)	Viewer
/graph/nodes/{id}/forest	GET	Get full link forest for all nodes owned by this identity	Owner
/graph/nodes/{id}/ego	GET	Get ego network (depth 1+2) centered on this node	Viewer
/graph/edges	POST	Create HypergraphEdge (triggers consent flow if required)	Contributor
/graph/edges/{id}	PATCH	Update edge attributes (weight, attributes, consent_config)	Owner(edge)
/graph/edges/{id}	DELETE	Delete edge (triggers shadow cell disconnection protocol)	Owner(edge)
/graph/shadow-cells	GET	List all shadow cells in this Grid from linked external Grids	Viewer

Endpoint	Method	Description	Min Auth
/graph/shadow-cells/{id}/sync	POST	Manually trigger MirrorAttribute sync from source entity	Editor
/graph/path	GET	Shortest path between two nodes (Dijkstra with edge weight)	Viewer
/graph/clusters	GET	Community cluster detection (Louvain algorithm) on Space subgraph	Manager
/graph/centrality	GET	Compute betweenness, closeness, eigenvector centrality	Manager
/federation/peers	GET	List all federation peers for this Grid	Admin
/federation/peers	POST	Register a new federation peer (initiates handshake)	Admin
/federation/peers/{id}	DELETE	Remove a federation peer (graceful disconnect)	Admin
/federation/sync	POST	Trigger manual full delta sync with all peers	Admin
/federation/sync/{peer_id}	POST	Trigger manual sync with one specific peer	Admin
/federation/conflicts	GET	List unresolved CRDT conflicts for review	Editor
/federation/conflicts/{id}	POST	Resolve a CRDT conflict with a chosen resolution strategy	Editor

11.3 Spaces, Workspaces, and Namespace API

Endpoint	Method	Description	Min Auth
/spaces	GET	List all Spaces the caller is a member of	Viewer
/spaces	POST	Create a new Space	Contributor
/spaces/{id}	GET	Get Space details and member roster	Viewer
/spaces/{id}	PATCH	Update Space configuration (governance, settings, treasury)	Manager
/spaces/{id}/join	POST	Join a public Space or accept a membership invitation	Contributor
/spaces/{id}/members	GET	List Space members with roles and contribution weights	Member
/spaces/{id}/members/{user_id}	PATCH	Update member role (Steward+ permission required)	Manager

Endpoint	Method	Description	Min Auth
/spaces/{id}/members/{user_id}	DELETE	Remove member from Space	Manager
/spaces/{id}/governance/proposals	GET	List all governance proposals (open and resolved)	Member
/spaces/{id}/governance/proposal	POST	Submit a governance proposal	Manager
/spaces/{id}/governance/vote	POST	Cast vote on an open governance proposal	Member
/spaces/{id}/treasury	GET	Get treasury account status and transaction history	Member
/spaces/{id}/link-subgraph	GET	Get materialized SpaceLinkSubgraph (updated hourly)	Member
/spaces/{id}/health	GET	Get SpaceHealthEngine output (member activity, governance, etc.)	Member
/spaces/{id}/dissolve	POST	Initiate dissolution workflow (requires governance vote)	Owner
/workspaces	POST	Create a new Workspace within a Space	Editor
/workspaces/{id}	GET	Get current Workspace state and open cubes	Member
/workspaces/{id}	PATCH	Update Workspace config (sheets, layout, dim_context)	Owner(ws)
/workspaces/{id}/session	POST	Open a Workspace session (returns WebSocket URL + session_id)	Member
/workspaces/{id}/session	DELETE	Close active Workspace session	Member
/workspaces/{id}/dim-context	PUT	Set workspace-level DimKey defaults (applied across all open cubes)	Member
/ns/resolve	GET	Resolve a NamespacePath URI to its entity ID and type	Viewer
/ns	POST	Register a new namespace entry	Owner(parent ns)
/ns/{id}	PATCH	Rename or update namespace entry (creates alias for old path)	Owner(ns)
/ns/{id}/alias	POST	Add an additional path alias for this namespace entry	Owner(ns)
/ns/{id}/federate	POST	Replicate namespace entry to a federation peer Grid	Admin

11.4 AI and Intelligence API

Endpoint	Method	Description	Min Auth
/ai/compute	POST	Request Tier-2 AI-computed attribute values for a batch of entities	Editor
/ai/nl-query	POST	Translate natural language to HyperQL and execute. Returns results + generated HyperQL.	Member
/ai/anomalies	GET	List AI-detected anomalies in a cube or DimSlice (paginated)	Viewer
/ai/anomalies/{id}/resolve	POST	Mark an anomaly as acknowledged or resolved	Editor
/ai/engines	GET	List all registered AIEngineAdapter plugins and their status	Admin
/ai/engines/{id}/health	GET	Get health and performance metrics for a specific AI engine	Admin
/ai/writeback	POST	Direct WritebackService endpoint (AI engines only — System auth)	System
/ai/oba/query	POST	Kogi Oba AI: reactive query + proactive hints in one response	Member
/ai/oba/hints/{entity_id}	GET	Get pending Oba proactive hints for a specific entity	Member
/ai/oba/actions/{entity_id}	POST	Submit an Oba autonomous action for user approval	Owner

11.5 WebSocket and Streaming Channels

Channel	Purpose	Events Emitted
wss://.../cubes/{id}/stream	Real-time cell mutation stream for a Hypercube.	CellUpdated · RowCreated · RowArchived · DimAdded · AttrRegistered · AIWriteback · ConflictDetected
wss://.../workspaces/{id}/session	Workspace collaboration session. Cursor presence and co-editing sync.	CursorMoved · CellLocked · CellUnlocked · PresenceJoined · PresenceLeft · DimContextChanged · ObaHintReceived
wss://.../spaces/{id}/activity	Space activity stream: member events, governance, graph changes.	MemberJoined · ProposalCreated · VoteSubmitted · EdgeCreated · ShadowCellUpdated · TreasuryEvent
wss://.../graph/shadow-sync	Shadow cell delta sync stream from linked external Grids.	ShadowAttrUpdated · ShadowDisconnected · ShadowReconnected · ShadowDegraded

Channel	Purpose	Events Emitted
wss://.../crdt/federation/{peer_id}	CRDT federation gossip stream between two Grid peers.	CrdtOpBatch · VectorClockExchange · ConflictSurfaced · SchemaDelta · HeartbeatPong
wss://.../ai/oba/stream	Oba AI real-time annotation stream. Proactive hints as they are generated.	ObaAnnotation · ObaActionProposed · ObaQueryResult · ObaAnomalyAlert

Chapter 12 — Apapo System Runtime and Environment

12.1 Service Mesh Architecture

The Apapo platform is deployed as a polyglot microservice mesh. Services communicate via gRPC internally and expose REST/WebSocket externally via the api-gateway. All internal communication uses mTLS. Service discovery uses Kubernetes-native DNS with Consul for cross-cluster cases.

Service	Language	Protocol	Port	Responsibility
api-gateway	Go	HTTP/2 + gRPC-Web + WSS	443/8080	Auth, rate limiting, routing, CORS, WebSocket upgrade, API versioning, request ID injection
portfolio-service	Go	gRPC + REST	9001	Kogi KPMS/KPSS: Component CRUD, sheet queries, PQL execution, computation orchestration
spreadsheet-service	Go	gRPC + REST	9002	HyperQL execution, DimSlice/DimFold/DimExpand evaluation, HypercubeView management
space-service	Go	gRPC + REST	9003	Space lifecycle, member management, governance config, treasury linking, SpaceLinkSubgraph
workspace-service	Go	gRPC + WebSocket	9004	Workspace sessions, sheet arrangements, DimContext management, presence, session history
link-network-service	Go+Rust	gRPC + REST	9005	KLNK: LinkEdge CRUD, consent flows, ShadowRow provisioning, graph traversal API
crdt-service	Rust	gRPC	9006	CrdtLog management, VectorClock advancement, apply_crdt_op, conflict detection
identity-service	Go	gRPC + REST	9007	SovereignEntity, Identity, Profile, Account management; VisibilityMask; CrossIdentityMerge
namespace-service	Go	gRPC + REST	9008	NamespacePath resolution, alias management, federation namespace sync, health reporting
writeback-service	Go	gRPC	9009	AI engine result ingestion, permission validation, audit trail, cell write dispatch
export-service	Go	REST (streaming)	9010	CSV, XLSX, JSON-LD, Parquet, Arrow export; webhook delivery; embed token generation
federation-coordinator	Go	gRPC + REST	9011	CrossGridLink consent flows; shadow sync routing; cross-Grid HyperQL fan-out; heartbeat

Service	Language	Protocol	Port	Responsibility
kogi-engine	Scala 3	gRPC + Kafka consumer	9100	All 21 kogi-engine sub-systems: analytics, risk, optimization, AI signals, MatchEngine
ume-engine	Scala 3	gRPC + Kafka consumer	9200	Ume AI: module health scoring, organizational analytics, RBAC evaluation assistance
qala-engine	Scala 3	gRPC + Kafka consumer	9300	Qala AI Agent: 10 capabilities — defect prediction, drift detection, security threat, etc.
hg-ops	Go	REST + Prometheus	9901	Distributed tracing (OpenTelemetry), metrics, structured logs, health probes, alerting

12.2 Kafka Topic Architecture

Apache Kafka is the central nervous system for all asynchronous operations in the Apapo platform: CRDT delta federation, AI engine event consumption, shadow cell sync, analytics pipeline, and WebSocket fan-out broadcast. The following topics are provisioned per Grid deployment.

Kafka Topic	Producers	Consumers	Partition Strategy	Retention
crdt.delta.{grid_id}	crdt-service (all nodes)	crdt-service (peer nodes)	Partition by cube_id for ordered delivery per cube	7 days
events.audit.{grid_id}	All services (dual-write)	kogi-engine, ume-engine, qala-engine, ClickHouse sink	Partition by component_id	30 days (hot); S3 archive indefinitely
ai.compute.{grid_id}	All services (on mutation)	kogi-engine, ume-engine, qala-engine worker pools	Partition by entity_id (collocate compute for same entity)	24 hours
ai.writeback.{grid_id}	kogi-engine, ume-engine, qala-engine	writeback-service	Partition by cube_id	24 hours
shadow_sync.{source_grid_id}	source grid cell mutations	host grid shadow sync consumers	Partition by shadow_cell_id	7 days
portfolio.actions.{grid_id}	portfolio-service	kogi-engine analytics, notification service	Partition by owner_entity_id	7 days
space.events.{grid_id}	space-service	space health engine, notification service, activity feed	Partition by space_id	7 days

Kafka Topic	Producers	Consumers	Partition Strategy	Retention
klmk.events.{grid_id}	link-network-service	KLNK graph engine, consent service	Partition by source_node_id	7 days
namespace.events.{grid_id}	namespace-service	namespace-service (all nodes) for cross-node cache invalidation	Partition by namespace prefix	7 days
hg.telemetry.{grid_id}	All services	hg-ops metrics collector, ClickHouse telemetry sink	Round-robin	24 hours

12.3 Observability Stack — HG-OPS

Capability	Implementation	Cardinality	Key Metrics / Traces
Distributed Tracing	OpenTelemetry SDK in all services. Traces exported to Jaeger or Tempo.	Full request traces across service boundaries	crdt_apply_duration_ms · query_plan_ms · ai_compute_ms · shadow_sync_latency_ms
Metrics	Prometheus scrape from /metrics on all services. Grafana dashboards.	Per-service + per-cube granularity	cell_mutations_per_sec · crdt_conflicts_per_min · federation_lag_ms · ai_queue_depth · ws_connections_active
Structured Logging	Zap (Go) / tracing (Rust) structured JSON logs. Forwarded to Loki or Elasticsearch.	Per-request log lines with trace_id	All auth decisions · all policy denials · all CRDT conflicts · all AI writeback completions
Health Probes	HTTP GET /health/live (liveness) and /health/ready (readiness) on all services.	Per-service probe per 10s	PostgreSQL connectivity · Redis connectivity · Kafka producer connectivity · federation peer heartbeats
Alerting	AlertManager rules on Prometheus metrics. PagerDuty integration for P0/P1.	Per-environment severity tiers	federation_lag > 30s · crdt_conflict_rate > 10/min · ai_compute_queue > 10K · snapshot_age > 48h
Audit Log	Append-only access_log table per service. Queryable via /audit API.	Per-request for write operations	actor · operation · entity_id · permission_decision · outcome · duration_ms

12.4 Environment Configuration Reference

```
# Apapo Grid deployment configuration (environment variables)
```

```

# — Core Grid identity —————
GRID_ID=<uuid> # Globally unique Grid identifier
GRID_NAME=kogi-production # Human-readable name
GRID_VERSION=1.0.0 # Hypergrid substrate version
NODE_ID=node-us-east-1 # This node's VectorClock identifier

# — PostgreSQL (primary cell store) —————
POSTGRES_URL=postgresql://user:pass@host:5432/hypergrid
POSTGRES_MAX_CONNECTIONS=100
POSTGRES_PARTITION_COUNT=32 # Hash partitions for components table

# — Redis Cluster (CRDT buffer + cache) —————
REDIS_CLUSTER_URLS=redis://host1:6379,redis://host2:6379
REDIS_CRDT_TTL_SECONDS=86400 # CrdtLog entries expire after 24h in Redis
REDIS_CELL_CACHE_DEFAULT_TTL=1800 # Default computed attr cache TTL

# — ClickHouse (analytics backend) —————
CLICKHOUSE_URL=http://host:8123/hypergrid_analytics
CLICKHOUSE_ANALYTICS_THRESHOLD=1000000 # Route FOLD queries > 1M rows to
ClickHouse

# — Apache Kafka (event streaming) —————
KAFKA_BOOTSTRAP_SERVERS=broker1:9092,broker2:9092
KAFKA_CRDT_TOPIC_PARTITIONS=32
KAFKA_REPLICATION_FACTOR=3
KAFKA_RETENTION_MS=604800000 # 7 days default retention

# — S3 / Object Store (EventLog archive + snapshots) —————
S3_BUCKET=hypergrid-event-archive
S3_REGION=us-east-1
S3_SNAPSHOT_BUCKET=hypergrid-snapshots
EVENTLOG_HOT_CAP=10000 # Flush to S3 after 10K events per entity

# — AI Engine —————
AI_ENGINE_MODE=openai_compatible # openai_compatible|local_llm|disabled
OPENAI_COMPATIBLE_BASE_URL=https://api.openai.com/v1
OPENAI_COMPATIBLE_API_KEY=<key>
LOCAL_LLM_URL=http://ollama:11434 # Used when AI_ENGINE_MODE=local_llm
AI_COMPUTE_WORKER_POOL_SIZE=16 # Concurrent Tier-2 compute workers

# — Federation —————
FEDERATION_ENABLED=true
FEDERATION_HEARTBEAT_INTERVAL_SECS=30
FEDERATION_MAX_PEERS=255
FEDERATION_TRUST_LEVEL=grid # grid|namespace|identity|all_three

# — Encryption —————
KMS_PROVIDER=aws_kms # aws_kms|hashicorp_vault|local_dev
KMS_KEY_ARN=arn:aws:kms:... # For aws_kms
VAULT_ADDR=http://vault:8200 # For hashicorp_vault
FIELD_ENCRYPTION_ENABLED=true # Field-level AES-256 for PII attrs

# — Observability —————
OTEL_EXPORTER_OTLP_ENDPOINT=http://jaeger:4317
PROMETHEUS_METRICS_PORT=9901
LOG_LEVEL=info # debug|info|warn|error
LOG_FORMAT=json # json|text

# — Domain system selector —————
DOMAIN_SYSTEM=kogi # kogi|ume|qala|hypergrid_standalone
KOGI_ENGINE_GRPC_ADDR=kogi-engine:9100

```

```
UME_ENGINE_GRPC_ADDR=ume-engine:9200
QALA_ENGINE_GRPC_ADDR=qala-engine:9300
```

12.5 Startup Bootstrap Sequence

Every Grid node follows a strict startup sequence that ensures the cell store, plugin registry, domain schemas, and federation connections are all fully initialized before accepting any requests. The sequence is idempotent — partial failures can be retried from any step.

Step	Service	Operation	Blocking?	Failure Handling
1. Config load	All	Load and validate all environment variables and config files. Crash-fail on missing required config.	Yes	Fatal: refuse to start. Log all missing config keys.
2. Storage connectivity	All	Verify PostgreSQL, Redis, and Kafka connectivity. Apply pending database migrations (Flyway/Liquibase).	Yes	Retry with exponential backoff. Alert after 60s. Fatal after 5min.
3. Plugin registry init	crdt-service	Load and health-check all HypercubePlugin implementations. Call <code>on_load()</code> for each. Log any Unhealthy plugins.	Yes	Degraded plugins: skip and log warning. Missing required plugins: fatal.
4. Grid bootstrap	spreadsheet-service	Call <code>DomainStore::bootstrap()</code> for all registered domain systems. Register Hypercubes and AttributeKeyDefs.	Yes	Idempotent: re-registering existing cube/attr is a no-op. Conflicts fatal.
5. EventLog recovery	crdt-service	Scan unapplied <code>CrdtLog</code> entries (<code>applied=false</code>). Re-apply in causal order. Advance <code>VectorClock</code> to current state.	Yes	On conflict: apply CRDT merge as normal. Log recovery events.
6. Namespace warm-up	namespace-service	Pre-populate Redis namespace resolution cache for all entries with <code>last_resolved_at</code> in last 7 days.	No (async)	Failure: cache starts cold. Resolution still works via PostgreSQL fallback.
7. Federation handshake	federation-coordinator	Initiate handshake with all registered federation peers. Run schema sync and delta sync for any diverged <code>VectorClocks</code> .	No (async)	Failure: node operates standalone. Federation retried every 30s.
8. AI engine warm-up	writeback-service	Health-check all <code>AIEngineAdapter</code> plugins. Pre-load model context for high-priority compute models.	No (async)	Failure: AI compute degrades gracefully. Cache serves stale values.

Step	Service	Operation	Blocki ng?	Failure Handling
9. Ready signal	All	Set readiness probe = healthy. api-gateway begins routing traffic to this node.	N/A	Only reached if steps 1–5 all succeeded.

End of Document

Apapo SDD Part II — Deep Implementation Reference · v1.0 · March 2026

Companion to Apapo SDD Part I: System Overview, Domain Operating Systems, Deployment Topology

Confidential — Internal Use Only